

Relocation Bonus

Attacking the Windows Loader Makes Analysts Switch Careers

Introduction

Introduction

Nick Cano

Introduction

Nick Cano

- 25 years old

Introduction

Nick Cano

- 25 years old
- Senior Security Architect at Cylance

Introduction

Nick Cano

- 25 years old
- Senior Security Architect at Cylance
- Author of *Game Hacking: Developing Autonomous Bots for Online Games*

Introduction

Nick Cano

- 25 years old
- Senior Security Architect at Cylance
- Author of *Game Hacking: Developing Autonomous Bots for Online Games*
- Pluralsight Instructor, *Modern C++ Secure Coding Practices: Const Correctness*

Introduction

Nick Cano

- 25 years old
- Senior Security Architect at Cylance
- Author of *Game Hacking: Developing Autonomous Bots for Online Games*
- Pluralsight Instructor, *Modern C++ Secure Coding Practices: Const Correctness*

Relocation Bonus

Introduction

Nick Cano

- 25 years old
- Senior Security Architect at Cylance
- Author of *Game Hacking: Developing Autonomous Bots for Online Games*
- Pluralsight Instructor, *Modern C++ Secure Coding Practices: Const Correctness*

Relocation Bonus

- A look into the Windows Portable Executable (PE) header and how it can be weaponized to *destroy* parsers, disassemblers, and other tools

Introduction

Nick Cano

- 25 years old
- Senior Security Architect at Cylance
- Author of *Game Hacking: Developing Autonomous Bots for Online Games*
- Pluralsight Instructor, *Modern C++ Secure Coding Practices: Const Correctness*

Relocation Bonus

- A look into the Windows Portable Executable (PE) header and how it can be weaponized to *destroy* parsers, disassemblers, and other tools
- A PE rebuilder that takes any 32bit PE then obfuscates and rebuilds it using the attack

Why Attack Relocations?

Why Attack Relocations?

00219E95	E8 8692F2FF	CALL Tibia.00143110	
00219E96	68 4B296200	PUSH Tibia.00692A48	
00219E97	8085 40FFFFFF	LEA EAX,DWORD PTR SS:[EBP-C0]	
00219E98	50	PUSH EAX	
00219E99	E8 3A202B00	CALL Tibia.004CC805	
00219E9A	CC	INT3	
00219E9B	CC	INT3	
00219E9C	CC	INT3	
00219E9D	CC	INT3	
00219E9E	CC	INT3	
00219E9F	CC	INT3	
00219EA0	55	PUSH EBP	
00219EA1	8BEC	MOV EBP,ESP	
00219EA2	6A FF	PUSH -1	
00219EA3	68 FA355400	PUSH Tibia.005435FA	
00219EA4	64:R1 00000000	MOV EAX,DWORD PTR FS:[0]	
00219EA5	50	PUSH EAX	
00219EA6	81EC 00010000	SUB ESP,100	
00219EA7	R1 80836200	MOV EAX,DWORD PTR DS:[698380]	
00219EA8	30C5	XOR EAX,EBP	
00219EA9	8945 EC	MOV DWORD PTR SS:[EBP-14],EAX	
00219EAB	53	PUSH EBX	
00219EAC	56	PUSH ESI	
00219EAD	57	PUSH EDI	
00219EAE	50	PUSH EAX	
00219EAF	8D45 F4	LEA EAX,DWORD PTR SS:[EBP-C]	
00219EB0	64:R3 00000000	MOV DWORD PTR FS:[0],EAX	
00219EB1	8965 F0	MOV DWORD PTR SS:[EBP-10],ESP	
00219EB2	8BF9	MOV EDI,ECX	
00219EB3	C745 FC 00000000	MOV DWORD PTR SS:[EBP-4],0	
00219EB4	BE 29008500	MOV ESI,Tibia.00858029	
00219EB5	89B5 00FFFFFF	MOV DWORD PTR SS:[EBP-F0],ESI	
00219EB6	83FF 03	CMF EDI,3	
00219EB7	0F87 C6000000	JM Tibia.00219FB4	
00219EB8	FF24B0 F0012100	JMP DWORD PTR DS:[EDI*4+21A1F0]	
00219EB9	BE 00245700	MOV ESI,Tibia.00672440	ASCII "You are using an outdated client which is no longer supp
00219EBA	89B5 00FFFFFF	MOV DWORD PTR SS:[EBP-F0],ESI	
00219EBB	B8 88258700	MOV EAX,Tibia.00672588	ASCII "Error"
00219EBC	EB 10	JMP SHORT Tibia.00219F17	
00219EBD	BE 00238700	MOV ESI,Tibia.00672380	ASCII "You are using an outdated client which is no longer supp
00219EBE	89B5 00FFFFFF	MOV DWORD PTR SS:[EBP-F0],ESI	
00219EBF	B8 94245700	MOV EAX,Tibia.00672454	ASCII "Information"
00219EC0	50	PUSH EAX	
00219EC1	8D40 8C	LEA ECX,DWORD PTR SS:[EBP-44]	
00219EC2	E8 4086F2FF	CALL Tibia.00138468	
00219EC3	C645 FC 01	MOV BYTE PTR SS:[EBP-4],1	
00219EC4	83FF 03	CMF EDI,3	
00219EC5	74 0A	JM SHORT Tibia.00219F33	
00219EC6	83FF 01	CMF EDI,1	
00219EC7	B9 5C7E7200	MOV ECX,Tibia.00707FEC	
00219EC8	75 05	JM SHORT Tibia.00219F38	
00219EC9	B9 942E7200	MOV ECX,Tibia.00707F94	
00219ECA	8B01	MOV EAX,DWORD PTR DS:[ECX]	
00219ECB	8945 08	MOV DWORD PTR SS:[EBP-28],EAX	
00219ECC	8B41 04	MOV EAX,DWORD PTR DS:[ECX+4]	
00219ECD	8945 0C	MOV DWORD PTR SS:[EBP-24],EAX	
00219ECE	8B41 08	MOV EAX,DWORD PTR DS:[ECX+8]	
00219ECF	8945 08	MOV DWORD PTR SS:[EBP-20],EAX	
00219ED0	8B41 0C	MOV EAX,DWORD PTR DS:[ECX+C]	
00219ED1	8945 E4	MOV DWORD PTR SS:[EBP-1C],EAX	
00219ED2	8B41 10	MOV EAX,DWORD PTR DS:[ECX+10]	

Why Attack Relocations?

00219E8F	E8 0692F2FF	CALL Tlbia.00143110	00219E8F	E8 0692F2FF	CALL Tlbia.00143110	
00219E90	68 48206900	PUSH Tlbia.00692048	00219E90	68 48206900	PUSH Tlbia.00692048	
00219E91	80B5 40FFFFF	LEA ERX,DMWORD PTR SS:[EBP-C0]	00219E91	80B5 40FFFFF	LEA ERX,DMWORD PTR SS:[EBP-C0]	
00219E92	50	PUSH ERX	00219E92	50	PUSH ERX	
00219E93	E8 3A202B00	CALL Tlbia.004CCB05	00219E93	E8 3A202B00	CALL Tlbia.004CCB05	
00219E94	50	INT3	00219E94	50	INT3	
00219E95	CC	INT3	00219E95	CC	INT3	
00219E96	CC	INT3	00219E96	CC	INT3	
00219E97	CC	INT3	00219E97	CC	INT3	
00219E98	55	PUSH EBP	00219E98	55	PUSH EBP	
00219E99	8BEC	MOV EBP,ESP	00219E99	8BEC	MOV EBP,ESP	
00219EA0	6A FF	PUSH -1	00219EA0	6A FF	PUSH -1	
00219EA1	68 FA355400	PUSH Tlbia.005435FA	00219EA1	68 FA355400	PUSH Tlbia.005435FA	
00219EA2	64:R1 00000000	MOV ERX,DMWORD PTR FS:[0]	00219EA2	64:R1 00000000	MOV ERX,DMWORD PTR FS:[0]	
00219EA3	50	PUSH ERX	00219EA3	50	PUSH ERX	
00219EB1	91EC 00010000	SUB ESP,100	00219EB1	91EC 00010000	SUB ESP,100	
00219EB7	R1 80836900	MOV ERX,DMWORD PTR DS:[698380]	00219EB7	R1 80836900	MOV ERX,DMWORD PTR DS:[698380]	
00219EB8	50	PUSH ERX	00219EB8	50	PUSH ERX	
00219EBE	8945 EC	MOV DMWORD PTR SS:[EBP-14],ERX	00219EBE	8945 EC	MOV DMWORD PTR SS:[EBP-14],ERX	
00219EC1	53	PUSH EBX	00219EC1	53	PUSH EBX	
00219EC2	56	PUSH ESI	00219EC2	56	PUSH ESI	
00219EC3	57	PUSH EDI	00219EC3	57	PUSH EDI	
00219EC4	50	PUSH ERX	00219EC4	50	PUSH ERX	
00219EC5	8D45 F4	LEA ERX,DMWORD PTR SS:[EBP-C]	00219EC5	8D45 F4	LEA ERX,DMWORD PTR SS:[EBP-C]	
00219EC6	64:R0 00000000	MOV DMWORD PTR FS:[0],ERX	00219EC6	64:R0 00000000	MOV DMWORD PTR FS:[0],ERX	
00219EC7	8965 F0	MOV DMWORD PTR SS:[EBP-10],ESP	00219EC7	8965 F0	MOV DMWORD PTR SS:[EBP-10],ESP	
00219ED1	8BF9	MOV EDI,ECX	00219ED1	8BF9	MOV EDI,ECX	
00219ED3	C745 FC 00000000	MOV DMWORD PTR SS:[EBP-4],0	00219ED3	C745 FC 00000000	MOV DMWORD PTR SS:[EBP-4],0	
00219ED4	BE 29085000	MOV ESI,Tlbia.00085029	00219ED4	BE 29085000	MOV ESI,Tlbia.00085029	
00219EDF	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	00219EDF	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	
00219EE5	83FF 03	CMP EDI,3	00219EE5	83FF 03	CMP EDI,3	
00219EE8	0F87 C6000000	JG Tlbia.00219F04	00219EE8	0F87 C6000000	JG Tlbia.00219F04	
00219EEC	FF24D0 F0012100	JMP DMWORD PTR DS:[EDI*4+21A1F0]	00219EEC	FF24D0 F0012100	JMP DMWORD PTR DS:[EDI*4+21A1F0]	
00219EF5	BE 80248700	MOV ESI,Tlbia.00072480	00219EF5	BE 80248700	MOV ESI,Tlbia.00072480	
00219EF6	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	00219EF6	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	
00219F00	B8 8B258700	MOV EAX,Tlbia.00072588	00219F00	B8 8B258700	MOV EAX,Tlbia.00072588	
00219F05	EB 10	JMP SHORT Tlbia.00219F17	00219F05	EB 10	JMP SHORT Tlbia.00219F17	
00219F07	BE 80238700	MOV ESI,Tlbia.00072380	00219F07	BE 80238700	MOV ESI,Tlbia.00072380	
00219F0C	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	00219F0C	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	
00219F12	B8 24248700	MOV ERX,Tlbia.00072494	00219F12	B8 24248700	MOV ERX,Tlbia.00072494	
00219F17	50	PUSH ERX	00219F17	50	PUSH ERX	
00219F19	8D40 BC	LEA ERX,DMWORD PTR SS:[EBP-44]	00219F19	8D40 BC	LEA ERX,DMWORD PTR SS:[EBP-44]	
00219F1B	E8 4006F2FF	CALL Tlbia.00139060	00219F1B	E8 4006F2FF	CALL Tlbia.00139060	
00219F20	C645 FC 01	MOV BYTE PTR SS:[EBP-4],1	00219F20	C645 FC 01	MOV BYTE PTR SS:[EBP-4],1	
00219F24	83FF 03	CMP EDI,3	00219F24	83FF 03	CMP EDI,3	
00219F27	74 0A	JC SHORT Tlbia.00219F33	00219F27	74 0A	JC SHORT Tlbia.00219F33	
00219F29	83FF 01	CMP EDI,1	00219F29	83FF 01	CMP EDI,1	
00219F2C	B9 6C7E7000	MOV ECX,Tlbia.000707F5C	00219F2C	B9 6C7E7000	MOV ECX,Tlbia.000707F5C	
00219F31	75 05	JNZ SHORT Tlbia.00219F39	00219F31	75 05	JNZ SHORT Tlbia.00219F39	
00219F33	B9 847E7000	MOV ECX,Tlbia.000707F84	00219F33	B9 847E7000	MOV ECX,Tlbia.000707F84	
00219F38	8B01	MOV ERX,DMWORD PTR DS:[ECX]	00219F38	8B01	MOV ERX,DMWORD PTR DS:[ECX]	
00219F3A	8945 D9	MOV DMWORD PTR SS:[EBP-20],ERX	00219F3A	8945 D9	MOV DMWORD PTR SS:[EBP-20],ERX	
00219F3C	8B41 04	MOV DMWORD PTR DS:[EBP-20],ERX	00219F3C	8B41 04	MOV DMWORD PTR DS:[EBP-20],ERX	
00219F40	8945 DC	MOV DMWORD PTR DS:[EBP-20],ERX	00219F40	8945 DC	MOV DMWORD PTR DS:[EBP-20],ERX	
00219F43	8B41 08	MOV ERX,DMWORD PTR DS:[EBP-20],ERX	00219F43	8B41 08	MOV ERX,DMWORD PTR DS:[EBP-20],ERX	
00219F46	8945 E0	MOV DMWORD PTR DS:[EBP-20],ERX	00219F46	8945 E0	MOV DMWORD PTR DS:[EBP-20],ERX	
00219F49	8B41 0C	MOV ERX,DMWORD PTR DS:[EBP-20],ERX	00219F49	8B41 0C	MOV ERX,DMWORD PTR DS:[EBP-20],ERX	
00219F4C	8945 E4	MOV DMWORD PTR DS:[EBP-20],ERX	00219F4C	8945 E4	MOV DMWORD PTR DS:[EBP-20],ERX	
00219F4F	8B41 10	MOV ERX,DMWORD PTR DS:[EBP-20],ERX	00219F4F	8B41 10	MOV ERX,DMWORD PTR DS:[EBP-20],ERX	

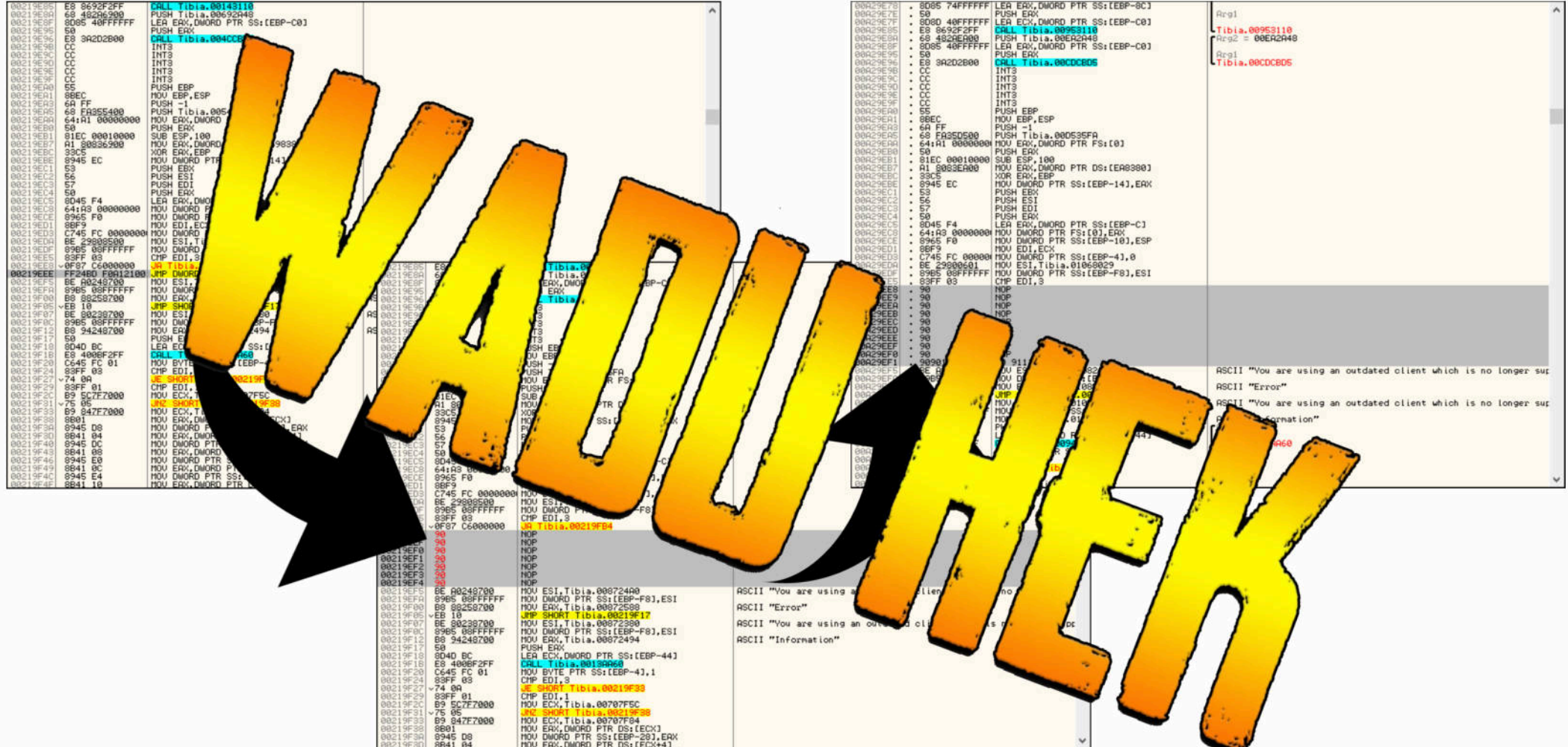
00219E8F	E8 0692F2FF	CALL Tlbia.00143110	00219E8F	E8 0692F2FF	CALL Tlbia.00143110	
00219E90	68 48206900	PUSH Tlbia.00692048	00219E90	68 48206900	PUSH Tlbia.00692048	
00219E91	80B5 40FFFFF	LEA ERX,DMWORD PTR SS:[EBP-C0]	00219E91	80B5 40FFFFF	LEA ERX,DMWORD PTR SS:[EBP-C0]	
00219E92	50	PUSH ERX	00219E92	50	PUSH ERX	
00219E93	E8 3A202B00	CALL Tlbia.004CCB05	00219E93	E8 3A202B00	CALL Tlbia.004CCB05	
00219E94	50	INT3	00219E94	50	INT3	
00219E95	CC	INT3	00219E95	CC	INT3	
00219E96	CC	INT3	00219E96	CC	INT3	
00219E97	CC	INT3	00219E97	CC	INT3	
00219E98	55	PUSH EBP	00219E98	55	PUSH EBP	
00219E99	8BEC	MOV EBP,ESP	00219E99	8BEC	MOV EBP,ESP	
00219EA0	6A FF	PUSH -1	00219EA0	6A FF	PUSH -1	
00219EA1	68 FA355400	PUSH Tlbia.005435FA	00219EA1	68 FA355400	PUSH Tlbia.005435FA	
00219EA2	64:R1 00000000	MOV ERX,DMWORD PTR FS:[0]	00219EA2	64:R1 00000000	MOV ERX,DMWORD PTR FS:[0]	
00219EA3	50	PUSH ERX	00219EA3	50	PUSH ERX	
00219EB1	91EC 00010000	SUB ESP,100	00219EB1	91EC 00010000	SUB ESP,100	
00219EB7	R1 80836900	MOV ERX,DMWORD PTR DS:[698380]	00219EB7	R1 80836900	MOV ERX,DMWORD PTR DS:[698380]	
00219EB8	50	PUSH ERX	00219EB8	50	PUSH ERX	
00219EBE	8945 EC	MOV DMWORD PTR SS:[EBP-14],ERX	00219EBE	8945 EC	MOV DMWORD PTR SS:[EBP-14],ERX	
00219EC1	53	PUSH EBX	00219EC1	53	PUSH EBX	
00219EC2	56	PUSH ESI	00219EC2	56	PUSH ESI	
00219EC3	57	PUSH EDI	00219EC3	57	PUSH EDI	
00219EC4	50	PUSH ERX	00219EC4	50	PUSH ERX	
00219EC5	8D45 F4	LEA ERX,DMWORD PTR SS:[EBP-C]	00219EC5	8D45 F4	LEA ERX,DMWORD PTR SS:[EBP-C]	
00219EC6	64:R0 00000000	MOV DMWORD PTR FS:[0],ERX	00219EC6	64:R0 00000000	MOV DMWORD PTR FS:[0],ERX	
00219EC7	8965 F0	MOV DMWORD PTR SS:[EBP-10],ESP	00219EC7	8965 F0	MOV DMWORD PTR SS:[EBP-10],ESP	
00219ED1	8BF9	MOV EDI,ECX	00219ED1	8BF9	MOV EDI,ECX	
00219ED3	C745 FC 00000000	MOV DMWORD PTR SS:[EBP-4],0	00219ED3	C745 FC 00000000	MOV DMWORD PTR SS:[EBP-4],0	
00219ED4	BE 29085000	MOV ESI,Tlbia.00085029	00219ED4	BE 29085000	MOV ESI,Tlbia.00085029	
00219EDF	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	00219EDF	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	
00219EE5	83FF 03	CMP EDI,3	00219EE5	83FF 03	CMP EDI,3	
00219EE8	0F87 C6000000	JG Tlbia.00219F04	00219EE8	0F87 C6000000	JG Tlbia.00219F04	
00219EEC	FF24D0 F0012100	JMP DMWORD PTR DS:[EDI*4+21A1F0]	00219EEC	FF24D0 F0012100	JMP DMWORD PTR DS:[EDI*4+21A1F0]	
00219EF5	BE 80248700	MOV ESI,Tlbia.00072480	00219EF5	BE 80248700	MOV ESI,Tlbia.00072480	
00219EF6	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	00219EF6	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	
00219F00	B8 8B258700	MOV EAX,Tlbia.00072588	00219F00	B8 8B258700	MOV EAX,Tlbia.00072588	
00219F05	EB 10	JMP SHORT Tlbia.00219F17	00219F05	EB 10	JMP SHORT Tlbia.00219F17	
00219F07	BE 80238700	MOV ESI,Tlbia.00072380	00219F07	BE 80238700	MOV ESI,Tlbia.00072380	
00219F0C	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	00219F0C	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	
00219F12	B8 24248700	MOV ERX,Tlbia.00072494	00219F12	B8 24248700	MOV ERX,Tlbia.00072494	
00219F17	50	PUSH ERX	00219F17	50	PUSH ERX	
00219F19	8D40 BC	LEA ERX,DMWORD PTR SS:[EBP-44]	00219F19	8D40 BC	LEA ERX,DMWORD PTR SS:[EBP-44]	
00219F1B	E8 4006F2FF	CALL Tlbia.00139060	00219F1B	E8 4006F2FF	CALL Tlbia.00139060	
00219F20	C645 FC 01	MOV BYTE PTR SS:[EBP-4],1	00219F20	C645 FC 01	MOV BYTE PTR SS:[EBP-4],1	
00219F24	83FF 03	CMP EDI,3	00219F24	83FF 03	CMP EDI,3	
00219F27	74 0A	JC SHORT Tlbia.00219F33	00219F27	74 0A	JC SHORT Tlbia.00219F33	
00219F29	83FF 01	CMP EDI,1	00219F29	83FF 01	CMP EDI,1	
00219F2C	B9 6C7E7000	MOV ECX,Tlbia.000707F5C	00219F2C	B9 6C7E7000	MOV ECX,Tlbia.000707F5C	
00219F31	75 05	JNZ SHORT Tlbia.00219F39	00219F31	75 05	JNZ SHORT Tlbia.00219F39	
00219F33	B9 847E7000	MOV ECX,Tlbia.000707F84	00219F33	B9 847E7000	MOV ECX,Tlbia.000707F84	
00219F38	8B01	MOV ERX,DMWORD PTR DS:[ECX]	00219F38	8B01	MOV ERX,DMWORD PTR DS:[ECX]	
00219F3A	8945 D9	MOV DMWORD PTR SS:[EBP-20],ERX	00219F3A	8945 D9	MOV DMWORD PTR SS:[EBP-20],ERX	
00219F3C	8B41 04	MOV DMWORD PTR DS:[EBP-20],ERX	00219F3C	8B41 04	MOV DMWORD PTR DS:[EBP-20],ERX	

00219E8F	E8 0692F2FF	CALL Tlbia.00143110	00219E8F	E8 0692F2FF	CALL Tlbia.00143110	
00219E90	68 48206900	PUSH Tlbia.00692048	00219E90	68 48206900	PUSH Tlbia.00692048	
00219E91	80B5 40FFFFF	LEA ERX,DMWORD PTR SS:[EBP-C0]	00219E91	80B5 40FFFFF	LEA ERX,DMWORD PTR SS:[EBP-C0]	
00219E92	50	PUSH ERX	00219E92	50	PUSH ERX	
00219E93	E8 3A202B00	CALL Tlbia.004CCB05	00219E93	E8 3A202B00	CALL Tlbia.004CCB05	
00219E94	50	INT3	00219E94	50	INT3	
00219E95	CC	INT3	00219E95	CC	INT3	
00219E96	CC	INT3	00219E96	CC	INT3	
00219E97	CC	INT3	00219E97	CC	INT3	
00219E98	55	PUSH EBP	00219E98	55	PUSH EBP	
00219E99	8BEC	MOV EBP,ESP	00219E99	8BEC	MOV EBP,ESP	
00219EA0	6A FF	PUSH -1	00219EA0	6A FF	PUSH -1	
00219EA1	68 FA355400	PUSH Tlbia.005435FA	00219EA1	68 FA355400	PUSH Tlbia.005435FA	
00219EA2	64:R1 00000000	MOV ERX,DMWORD PTR FS:[0]	00219EA2	64:R1 00000000	MOV ERX,DMWORD PTR FS:[0]	
00219EA3	50	PUSH ERX	00219EA3	50	PUSH ERX	
00219EB1	91EC 00010000	SUB ESP,100	00219EB1	91EC 00010000	SUB ESP,100	
00219EB7	R1 80836900	MOV ERX,DMWORD PTR DS:[698380]	00219EB7	R1 80836900	MOV ERX,DMWORD PTR DS:[698380]	
00219EB8	50	PUSH ERX	00219EB8	50	PUSH ERX	
00219EBE	8945 EC	MOV DMWORD PTR SS:[EBP-14],ERX	00219EBE	8945 EC	MOV DMWORD PTR SS:[EBP-14],ERX	
00219EC1	53	PUSH EBX	00219EC1	53	PUSH EBX	
00219EC2	56	PUSH ESI	00219EC2	56	PUSH ESI	
00219EC3	57	PUSH EDI	00219EC3	57	PUSH EDI	
00219EC4	50	PUSH ERX	00219EC4	50	PUSH ERX	
00219EC5	8D45 F4	LEA ERX,DMWORD PTR SS:[EBP-C]	00219EC5	8D45 F4	LEA ERX,DMWORD PTR SS:[EBP-C]	
00219EC6	64:R0 00000000	MOV DMWORD PTR FS:[0],ERX	00219EC6	64:R0 00000000	MOV DMWORD PTR FS:[0],ERX	
00219EC7	8965 F0	MOV DMWORD PTR SS:[EBP-10],ESP	00219EC7	8965 F0	MOV DMWORD PTR SS:[EBP-10],ESP	
00219ED1	8BF9	MOV EDI,ECX	00219ED1	8BF9	MOV EDI,ECX	
00219ED3	C745 FC 00000000	MOV DMWORD PTR SS:[EBP-4],0	00219ED3	C745 FC 00000000	MOV DMWORD PTR SS:[EBP-4],0	
00219ED4	BE 29085000	MOV ESI,Tlbia.00085029	00219ED4	BE 29085000	MOV ESI,Tlbia.00085029	
00219EDF	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	00219EDF	8965 08FFFFFF	MOV DMWORD PTR SS:[EBP-F0],ESI	
00219EE5	83FF 03	CMP EDI,3	00219EE5	83FF 03	CMP EDI,3	
00219EE8	0F87 C6000000	JG Tlbia.00219F04	00219EE8	0F87 C6000000	JG Tlbia.00219F04	
00219EEC	FF24D0 F0012100	JMP DMWORD PTR DS:[EDI*4+21A1F0]	00219EEC	FF24D0 F0012100	JMP DMWORD PTR DS:[EDI*4+21A1F0]	
00219EF5	BE 802487					

Why Attack Relocations?

00219E50 00219E51 00219E52 00219E53 00219E54 00219E55 00219E56 00219E57 00219E58 00219E59 00219E60 00219E61 00219E62 00219E63 00219E64 00219E65 00219E66 00219E67 00219E68 00219E69 00219E70 00219E71 00219E72 00219E73 00219E74 00219E75 00219E76 00219E77 00219E78 00219E79 00219E80 00219E81 00219E82 00219E83 00219E84 00219E85 00219E86 00219E87 00219E88 00219E89 00219E90 00219E91 00219E92 00219E93 00219E94 00219E95 00219E96 00219E97 00219E98 00219E99 00219EA0 00219EA1 00219EA2 00219EA3 00219EA4 00219EA5 00219EA6 00219EA7 00219EA8 00219EA9 00219EAA 00219EAB 00219EAC 00219EAD 00219EAE 00219EAF 00219EB0 00219EB1 00219EB2 00219EB3 00219EB4 00219EB5 00219EB6 00219EB7 00219EB8 00219EB9 00219EBA 00219EBB 00219EBC 00219EBD 00219EBE 00219EBF 00219EC0 00219EC1 00219EC2 00219EC3 00219EC4 00219EC5 00219EC6 00219EC7 00219EC8 00219EC9 00219EDA 00219EDB 00219EDC 00219EDD 00219EDE 00219EDF 00219EE0 00219EE1 00219EE2 00219EE3 00219EE4 00219EE5 00219EE6 00219EE7 00219EE8 00219EE9 00219EEA 00219EEB 00219EEC 00219EED 00219EEE 00219EEF 00219EF0 00219EF1 00219EF2 00219EF3 00219EF4 00219EF5 00219EF6 00219EF7 00219EF8 00219EF9 00219F00 00219F01 00219F02 00219F03 00219F04 00219F05 00219F06 00219F07 00219F08 00219F09 00219F0A 00219F0B 00219F0C 00219F0D 00219F0E 00219F0F 00219F10 00219F11 00219F12 00219F13 00219F14 00219F15 00219F16 00219F17 00219F18 00219F19 00219F1A 00219F1B 00219F1C 00219F1D 00219F1E 00219F1F 00219F20 00219F21 00219F22 00219F23 00219F24 00219F25 00219F26 00219F27 00219F28 00219F29 00219F2A 00219F2B 00219F2C 00219F2D 00219F2E 00219F2F 00219F30 00219F31 00219F32 00219F33 00219F34 00219F35 00219F36 00219F37 00219F38 00219F39 00219F3A 00219F3B 00219F3C 00219F3D 00219F3E 00219F3F 00219F40 00219F41 00219F42 00219F43 00219F44 00219F45 00219F46 00219F47 00219F48 00219F49 00219F4A 00219F4B 00219F4C 00219F4D 00219F4E 00219F4F 00219F50 00219F51 00219F52 00219F53 00219F54 00219F55 00219F56 00219F57 00219F58 00219F59 00219F5A 00219F5B 00219F5C 00219F5D 00219F5E 00219F5F 00219F60 00219F61 00219F62 00219F63 00219F64 00219F65 00219F66 00219F67 00219F68 00219F69 00219F6A 00219F6B 00219F6C 00219F6D 00219F6E 00219F6F 00219F70 00219F71 00219F72 00219F73 00219F74 00219F75 00219F76 00219F77 00219F78 00219F79 00219F7A 00219F7B 00219F7C 00219F7D 00219F7E 00219F7F 00219F80 00219F81 00219F82 00219F83 00219F84 00219F85 00219F86 00219F87 00219F88 00219F89 00219F8A 00219F8B 00219F8C 00219F8D 00219F8E 00219F8F 00219F90 00219F91 00219F92 00219F93 00219F94 00219F95 00219F96 00219F97 00219F98 00219F99 00219FA0 00219FA1 00219FA2 00219FA3 00219FA4 00219FA5 00219FA6 00219FA7 00219FA8 00219FA9 00219FAA 00219FAB 00219FAC 00219FAD 00219FAE 00219FAF 00219FB0 00219FB1 00219FB2 00219FB3 00219FB4 00219FB5 00219FB6 00219FB7 00219FB8 00219FB9 00219FBA 00219FBB 00219FBC 00219FBD 00219FBE 00219FBF 00219FC0 00219FC1 00219FC2 00219FC3 00219FC4 00219FC5 00219FC6 00219FC7 00219FC8 00219FC9 00219FCA 00219FCB 00219FCC 00219FCD 00219FCE 00219FCF 00219FD0 00219FD1 00219FD2 00219FD3 00219FD4 00219FD5 00219FD6 00219FD7 00219FD8 00219FD9 00219FDA 00219FDB 00219FDC 00219FDD 00219FDE 00219FDF 00219FE0 00219FE1 00219FE2 00219FE3 00219FE4 00219FE5 00219FE6 00219FE7 00219FE8 00219FE9 00219FEA 00219FEB 00219FEC 00219FED 00219FEE 00219FEF 00219FF0 00219FF1 00219FF2 00219FF3 00219FF4 00219FF5 00219FF6 00219FF7 00219FF8 00219FF9 00219FFA 00219FFB 00219FFC 00219FFD 00219FFE 00219FFF 0021A000 0021A001 0021A002 0021A003 0021A004 0021A005 0021A006 0021A007 0021A008 0021A009 0021A00A 0021A00B 0021A00C 0021A00D 0021A00E 0021A00F 0021A010 0021A011 0021A012 0021A013 0021A014 0021A015 0021A016 0021A017 0021A018 0021A019 0021A01A 0021A01B 0021A01C 0021A01D 0021A01E 0021A01F 0021A020 0021A021 0021A022 0021A023 0021A024 0021A025 0021A026 0021A027 0021A028 0021A029 0021A02A 0021A02B 0021A02C 0021A02D 0021A02E 0021A02F 0021A030 0021A03

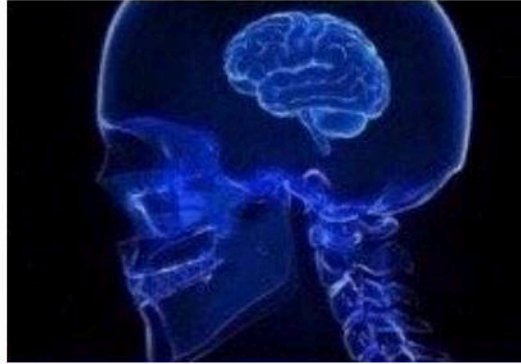
Why Attack Relocations?



Why Attack Relocations?

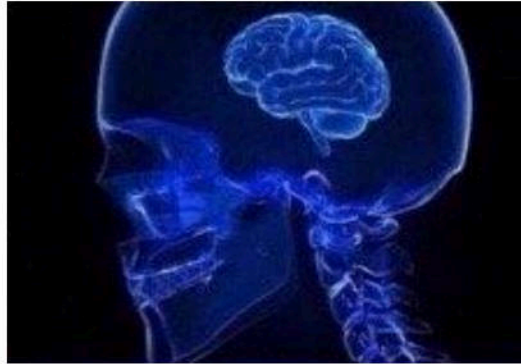
Why Attack Relocations?

its broken for no
reason

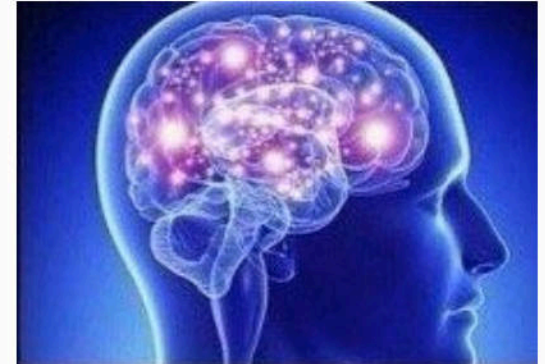


Why Attack Relocations?

its broken for no reason

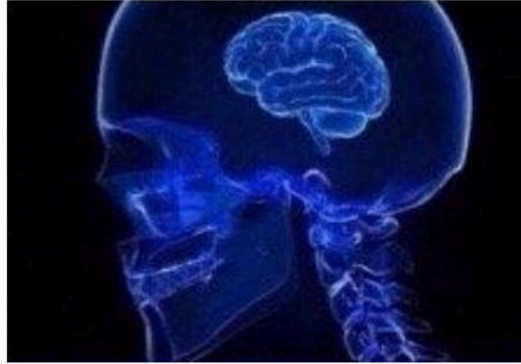


relocations corrupted the patched code

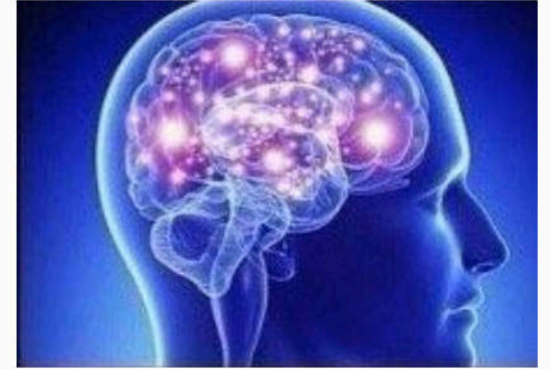


Why Attack Relocations?

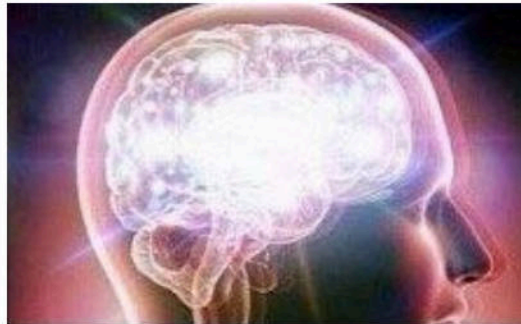
its broken for no reason



relocations corrupted the patched code

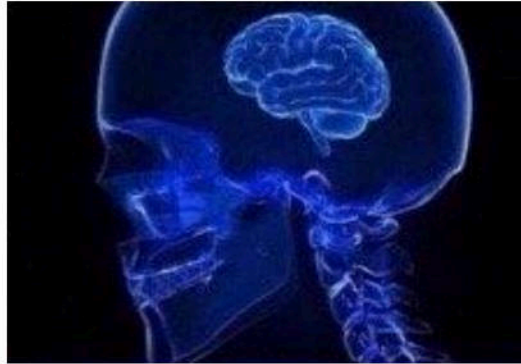


don't patch code that is relocated

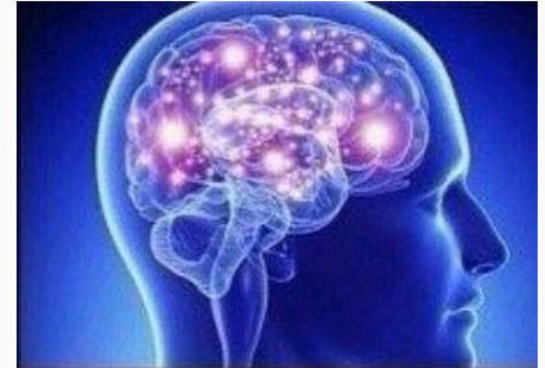


Why Attack Relocations?

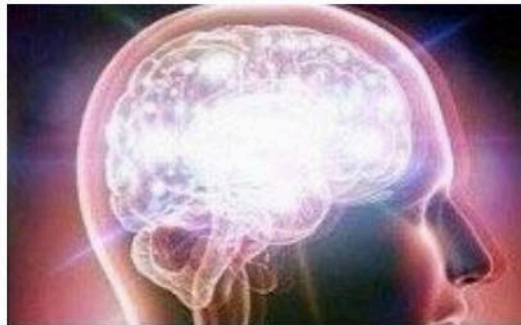
its broken for no reason



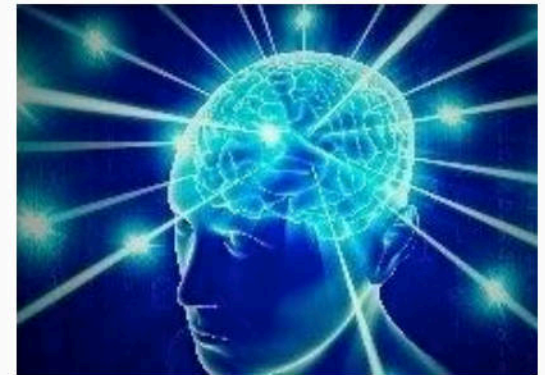
relocations corrupted the patched code



don't patch code that is relocated



relocations can be weaponized to hide my arsenal in the bowels of the machine



Mission Statement

Mission Statement

Me: *slaps roof of exe*
this bad boy can fit so many
reloc entries in it



What Are Relocations?

What Are Relocations?

Relocations exist to enable dynamic mapping, specifically ASLR

What Are Relocations?

Relocations exist to enable dynamic mapping, specifically ASLR

0x012E0000 Base

0133A00C	. AF9F3301	DD Example.01339FAF
0133A010	. D29F3301	DD Example.01339FD2
0133A014	. 00A03301	DD Example.0133A000
0133A018	. 00A03301	DD Example.0133A000
0133A01C	. 00A03301	DD Example.0133A000
0133A020	. 00A03301	DD Example.0133A000
0133A024	. 00A03301	DD Example.0133A000
0133A028	. 00A03301	DD Example.0133A000
0133A02C	. 00A03301	DD Example.0133A000
0133A030	. 00A03301	DD Example.0133A000
0133A034	. 00A03301	DD Example.0133A000
0133A038	. 00A03301	DD Example.0133A000
0133A03C	. 00A03301	DD Example.0133A000
0133A040	. 00A03301	DD Example.0133A000
0133A044	. 00A03301	DD Example.0133A000
0133A048	. 00A03301	DD Example.0133A000
0133A04C	. 00A03301	DD Example.0133A000
0133A050	. 00A03301	DD Example.0133A000
0133A054	. 00A03301	DD Example.0133A000
0133A058	. 00A03301	DD Example.0133A000

0x01120000 Base

0117A00C	. AF9F1701	DD Example.01179FAF
0117A010	. D29F1701	DD Example.01179FD2
0117A014	. 00A01701	DD Example.0117A000
0117A018	. 00A01701	DD Example.0117A000
0117A01C	. 00A01701	DD Example.0117A000
0117A020	. 00A01701	DD Example.0117A000
0117A024	. 00A01701	DD Example.0117A000
0117A028	. 00A01701	DD Example.0117A000
0117A02C	. 00A01701	DD Example.0117A000
0117A030	. 00A01701	DD Example.0117A000
0117A034	. 00A01701	DD Example.0117A000
0117A038	. 00A01701	DD Example.0117A000
0117A03C	. 00A01701	DD Example.0117A000
0117A040	. 00A01701	DD Example.0117A000
0117A044	. 00A01701	DD Example.0117A000
0117A048	. 00A01701	DD Example.0117A000
0117A04C	. 00A01701	DD Example.0117A000
0117A050	. 00A01701	DD Example.0117A000
0117A054	. 00A01701	DD Example.0117A000
0117A058	. 00A01701	DD Example.0117A000

What Are Relocations?

Relocations exist to enable dynamic mapping, specifically ASLR

0x012E0000 Base

0133A00C	. AF9F3301	DD Example. 01339FAF
0133A010	. D29F3301	DD Example. 01339FD2
0133A014	. 00A03301	DD Example. 0133A000
0133A018	. 00A03301	DD Example. 0133A000
0133A01C	. 00A03301	DD Example. 0133A000
0133A020	. 00A03301	DD Example. 0133A000
0133A024	. 00A03301	DD Example. 0133A000
0133A028	. 00A03301	DD Example. 0133A000
0133A02C	. 00A03301	DD Example. 0133A000
0133A030	. 00A03301	DD Example. 0133A000
0133A034	. 00A03301	DD Example. 0133A000
0133A038	. 00A03301	DD Example. 0133A000
0133A03C	. 00A03301	DD Example. 0133A000
0133A040	. 00A03301	DD Example. 0133A000
0133A044	. 00A03301	DD Example. 0133A000
0133A048	. 00A03301	DD Example. 0133A000
0133A04C	. 00A03301	DD Example. 0133A000
0133A050	. 00A03301	DD Example. 0133A000
0133A054	. 00A03301	DD Example. 0133A000
0133A058	. 00A03301	DD Example. 0133A000

0x01120000 Base

0117A00C	. AF9F1701	DD Example. 01179FAF
0117A010	. D29F1701	DD Example. 01179FD2
0117A014	. 00A01701	DD Example. 0117A000
0117A018	. 00A01701	DD Example. 0117A000
0117A01C	. 00A01701	DD Example. 0117A000
0117A020	. 00A01701	DD Example. 0117A000
0117A024	. 00A01701	DD Example. 0117A000
0117A028	. 00A01701	DD Example. 0117A000
0117A02C	. 00A01701	DD Example. 0117A000
0117A030	. 00A01701	DD Example. 0117A000
0117A034	. 00A01701	DD Example. 0117A000
0117A038	. 00A01701	DD Example. 0117A000
0117A03C	. 00A01701	DD Example. 0117A000
0117A040	. 00A01701	DD Example. 0117A000
0117A044	. 00A01701	DD Example. 0117A000
0117A048	. 00A01701	DD Example. 0117A000
0117A04C	. 00A01701	DD Example. 0117A000
0117A050	. 00A01701	DD Example. 0117A000
0117A054	. 00A01701	DD Example. 0117A000
0117A058	. 00A01701	DD Example. 0117A000

What Are Relocations?

Relocations exist to enable dynamic mapping, specifically ASLR

0x012E0000 Base

0133A00C	. AF9F3301	DD Example. 01339FAF
0133A010	. D29F3301	DD Example. 01339FD2
0133A014	. 00A03301	DD Example. 0133A000
0133A018	. 00A03301	DD Example. 0133A000
0133A01C	. 00A03301	DD Example. 0133A000
0133A020	. 00A03301	DD Example. 0133A000
0133A024	. 00A03301	DD Example. 0133A000
0133A028	. 00A03301	DD Example. 0133A000
0133A02C	. 00A03301	DD Example. 0133A000
0133A030	. 00A03301	DD Example. 0133A000
0133A034	. 00A03301	DD Example. 0133A000
0133A038	. 00A03301	DD Example. 0133A000
0133A03C	. 00A03301	DD Example. 0133A000
0133A040	. 00A03301	DD Example. 0133A000
0133A044	. 00A03301	DD Example. 0133A000
0133A048	. 00A03301	DD Example. 0133A000
0133A04C	. 00A03301	DD Example. 0133A000
0133A050	. 00A03301	DD Example. 0133A000
0133A054	. 00A03301	DD Example. 0133A000
0133A058	. 00A03301	DD Example. 0133A000

0x01120000 Base

0117A00C	. AF9F1701	DD Example. 01179FAF
0117A010	. D29F1701	DD Example. 01179FD2
0117A014	. 00A01701	DD Example. 0117A000
0117A018	. 00A01701	DD Example. 0117A000
0117A01C	. 00A01701	DD Example. 0117A000
0117A020	. 00A01701	DD Example. 0117A000
0117A024	. 00A01701	DD Example. 0117A000
0117A028	. 00A01701	DD Example. 0117A000
0117A02C	. 00A01701	DD Example. 0117A000
0117A030	. 00A01701	DD Example. 0117A000
0117A034	. 00A01701	DD Example. 0117A000
0117A038	. 00A01701	DD Example. 0117A000
0117A03C	. 00A01701	DD Example. 0117A000
0117A040	. 00A01701	DD Example. 0117A000
0117A044	. 00A01701	DD Example. 0117A000
0117A048	. 00A01701	DD Example. 0117A000
0117A04C	. 00A01701	DD Example. 0117A000
0117A050	. 00A01701	DD Example. 0117A000
0117A054	. 00A01701	DD Example. 0117A000
0117A058	. 00A01701	DD Example. 0117A000

What Are Relocations?

Relocations exist to enable dynamic mapping, specifically ASLR

0x012E0000 Base

0133A00C	. AF9F3301	DD Example.01339FAF
0133A010	. D29F3301	DD Example.01339FD2
0133A014	. 00A03301	DD Example.0133A000
0133A018	. 00A03301	DD Example.0133A000
0133A01C	. 00A03301	DD Example.0133A000
0133A020	. 00A03301	DD Example.0133A000
0133A024	. 00A03301	DD Example.0133A000
0133A028	. 00A03301	DD Example.0133A000
0133A02C	. 00A03301	DD Example.0133A000
0133A030	. 00A03301	DD Example.0133A000
0133A034	. 00A03301	DD Example.0133A000
0133A038	. 00A03301	DD Example.0133A000
0133A03C	. 00A03301	DD Example.0133A000
0133A040	. 00A03301	DD Example.0133A000
0133A044	. 00A03301	DD Example.0133A000
0133A048	. 00A03301	DD Example.0133A000
0133A04C	. 00A03301	DD Example.0133A000
0133A050	. 00A03301	DD Example.0133A000
0133A054	. 00A03301	DD Example.0133A000
0133A058	. 00A03301	DD Example.0133A000

0x01120000 Base

0117A00C	. AF9F1701	DD Example.01179FAF
0117A010	. D29F1701	DD Example.01179FD2
0117A014	. 00A01701	DD Example.0117A000
0117A018	. 00A01701	DD Example.0117A000
0117A01C	. 00A01701	DD Example.0117A000
0117A020	. 00A01701	DD Example.0117A000
0117A024	. 00A01701	DD Example.0117A000
0117A028	. 00A01701	DD Example.0117A000
0117A02C	. 00A01701	DD Example.0117A000
0117A030	. 00A01701	DD Example.0117A000
0117A034	. 00A01701	DD Example.0117A000
0117A038	. 00A01701	DD Example.0117A000
0117A03C	. 00A01701	DD Example.0117A000
0117A040	. 00A01701	DD Example.0117A000
0117A044	. 00A01701	DD Example.0117A000
0117A048	. 00A01701	DD Example.0117A000
0117A04C	. 00A01701	DD Example.0117A000
0117A050	. 00A01701	DD Example.0117A000
0117A054	. 00A01701	DD Example.0117A000
0117A058	. 00A01701	DD Example.0117A000

PE Header Sidebar

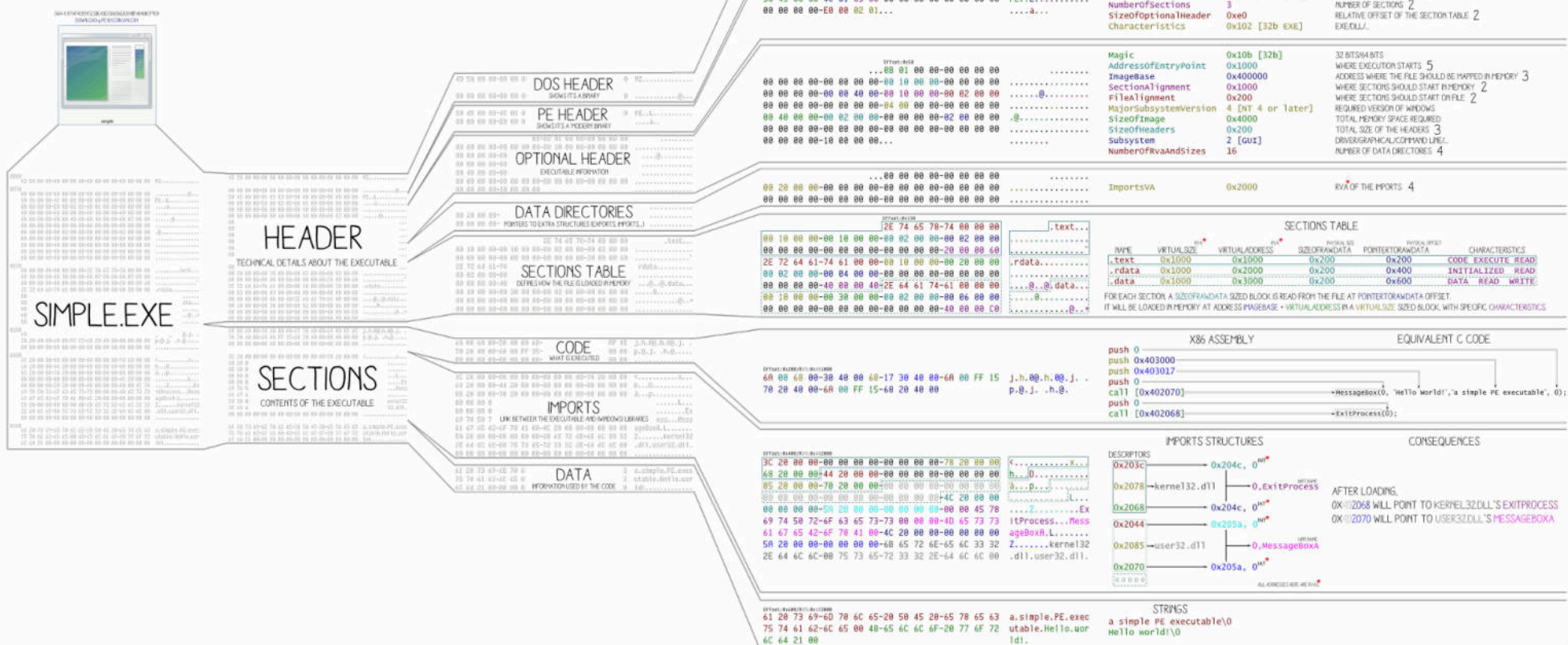
PE Header Sidebar

PE¹⁰¹ a windows executable walk-through

ANGE ALBERTINI

CORKAMI.COM

DISSECTED PE



THIS IS THE WHOLE FILE, HOWEVER, MOST OF FILES CONTAIN MORE ELEMENTS.
EXPLANATIONS ARE SHORTEST FOR CONCISENESS.

PE Header Sidebar

PE¹⁰¹ a windows executable walk-through

ANGE ALBERTINI
CORKAMI.COM

DISSECTED PE

ImageBase (what we'll call desiredBase) is where the binary requests to be mapped in memory.

SECTIONS TABLE

NAME	VIRTUAL SIZE	VIRTUAL ADDRESS	SIZE OF DATA	PORTER OR DATA	CHARACTERISTICS
.text	0x1000	0x1000	0x200	0x200	CODE, EXECUTE, READ
.data	0x1000	0x2000	0x400	0x400	INITIALIZED, READ, DATA, READ, WRITE
.@.data	0x1000	0x3000	0x200	0x400	DATA, READ, WRITE

IMPORTS STRUCTURES

DESCRIPTORS

- 0x203C → 0x204C, 0x0000
- 0x2078 → kernel32.dll
- 0x2068 → 0x204C, 0x0000
- 0x2044 → 0x205A, 0x0000
- 0x2085 → user32.dll
- 0x2070 → 0x205A, 0x0000

CONSEQUENCES

AFTER LOADING, 0x00000000 WILL POINT TO KERNEL32.DLL'S EXITPROCESS, 0x00000000 WILL POINT TO USER32.DLL'S MESSAGEBOXA

STRINGS

a simple PE executable
Hello world!

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS
EXPLANATIONS ARE PROVIDED FOR CONCISENESS

PE Header Sidebar

PE¹⁰¹ a windows executable walk-through

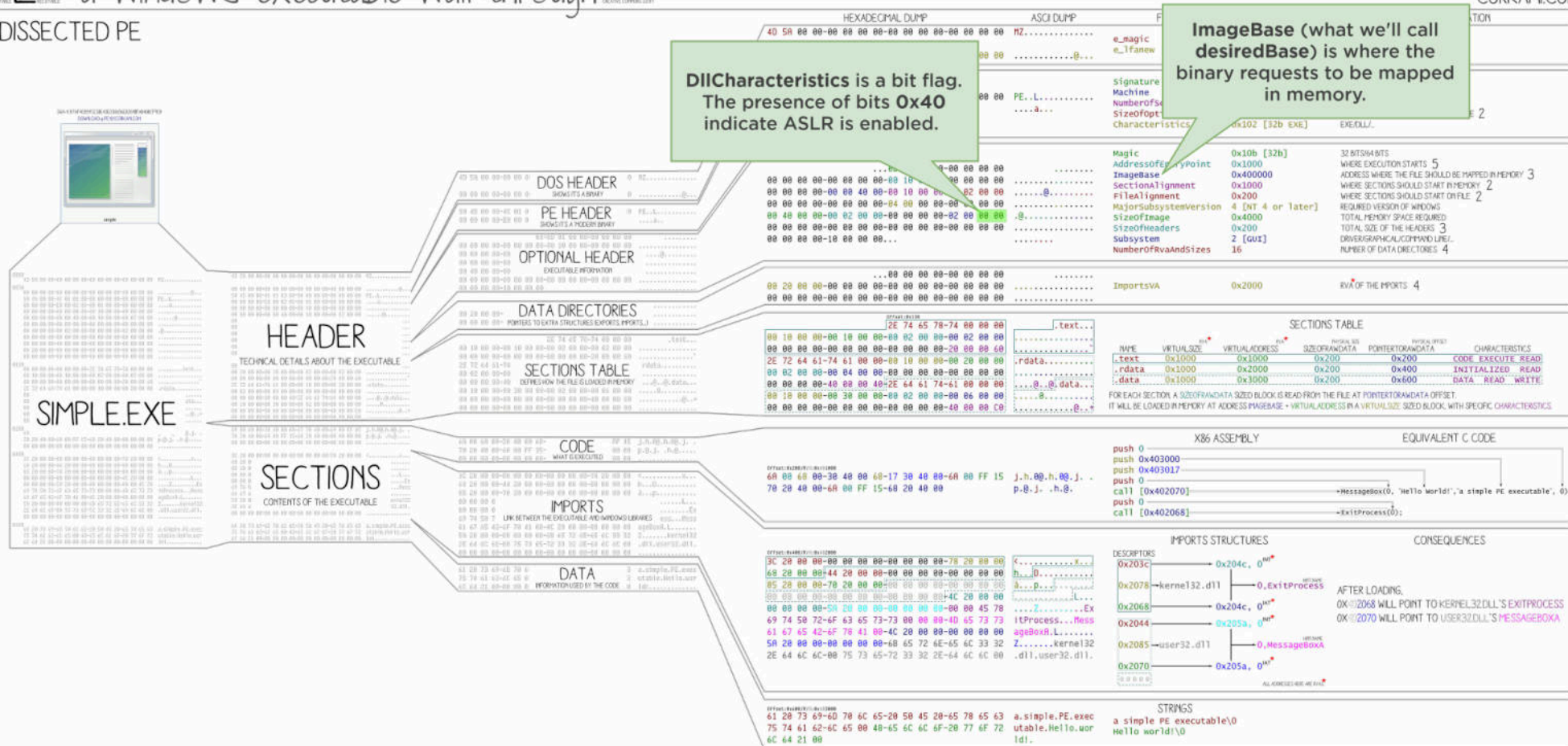
ANGE ALBERTINI

CORKAMI.COM

DISSECTED PE

DllCharacteristics is a bit flag.
The presence of bits **0x40**
indicate ASLR is enabled.

ImageBase (what we'll call **desiredBase**) is where the binary requests to be mapped in memory.



THIS IS THE WHOLE FILE, HOWEVER, MOST OF FILES CONTAIN MORE ELEMENTS
AND ANALYSIS ARE THEN MADE FOR CONCLUSIONS

PE Header Sidebar

PE¹⁰¹ a windows executable walk-through

ANGE ALBERTINI
CORKAMI.COM

DISSECTED PE

ImageBase (what we'll call desiredBase) is where the binary requests to be mapped in memory.

DllCharacteristics is a bit flag. The presence of bits 0x40 indicate ASLR is enabled.

The DataDirectory array stores information about the relocations table at index IMAGE_DIRECTORY_ENTRY_BASERELOC, typically pointing into a section named .reloc

SECTIONS TABLE

NAME	VIRTUAL SIZE	VIRTUAL ADDRESS	SIZE OF DATA	PORTER OR DATA	CHARACTERISTICS
.text	0x1000	0x1000	0x200	0x200	CODE, EXECUTE, READ
.data	0x1000	0x2000	0x200	0x400	INITIALIZED, READ, DATA, READ, WRITE

IMPORTS STRUCTURES

DESCRIPTORS	IMPORTS
0x203C	kernel32.dll
0x2078	user32.dll
0x2044	kernel32.dll
0x2085	user32.dll
0x2070	kernel32.dll

CONSEQUENCES

AFTER LOADING, 0x2068 WILL POINT TO KERNEL32.DLL'S EXITPROCESS
0x2070 WILL POINT TO USER32.DLL'S MESSAGEBOX

STRINGS

a simple PE executable
Hello world!

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MORE ELEMENTS
EXPLANATIONS ARE PROVIDED FOR CONCISENESS

How Do Relocations Work?

How Do Relocations Work?

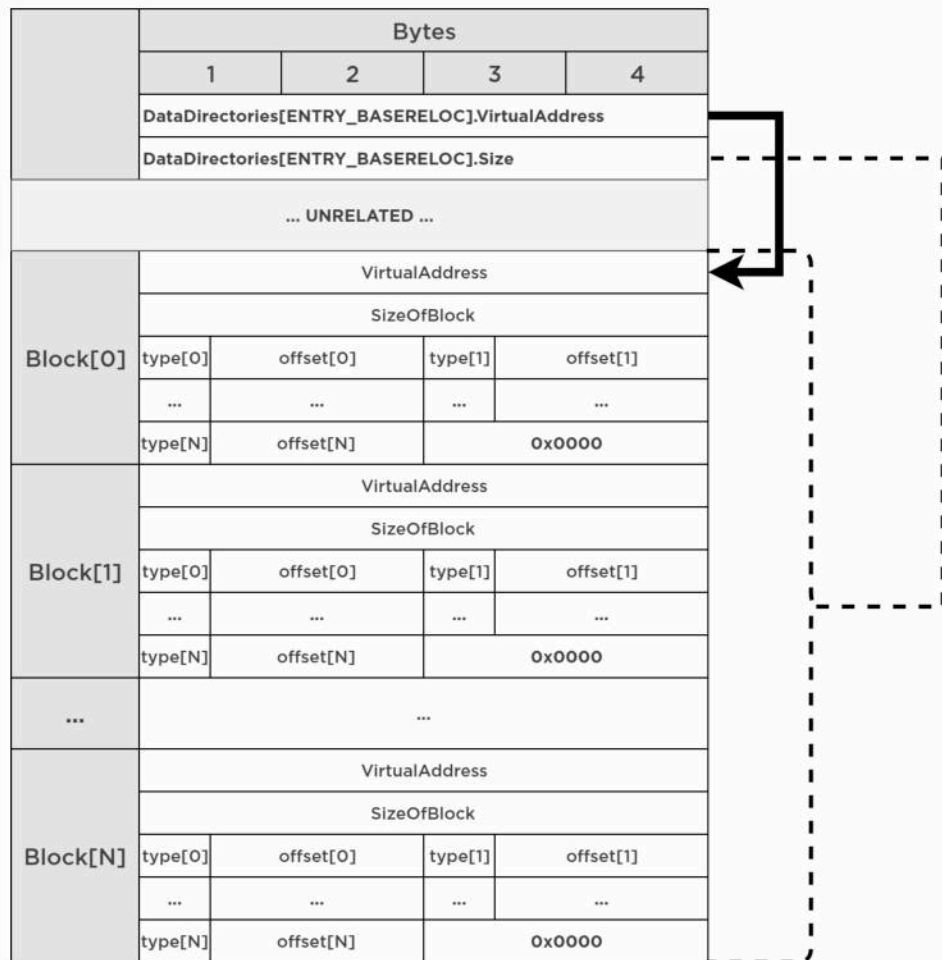
	Bytes			
	1	2	3	4
	DataDirectories[ENTRY_BASERELOC].VirtualAddress			
	DataDirectories[ENTRY_BASERELOC].Size			
... UNRELATED ...				
Block[0]	VirtualAddress			
	SizeOfBlock			
	type[0]	offset[0]	type[1]	offset[1]
	...	...	...	...
	type[N]	offset[N]	0x0000	
Block[1]	VirtualAddress			
	SizeOfBlock			
	type[0]	offset[0]	type[1]	offset[1]
	...	...	...	...
	type[N]	offset[N]	0x0000	
...	...			
Block[N]	VirtualAddress			
	SizeOfBlock			
	type[0]	offset[0]	type[1]	offset[1]
	...	...	...	...
	type[N]	offset[N]	0x0000	

How Do Relocations Work?

	Bytes			
	1	2	3	4
	DataDirectories[ENTRY_BASERELOC].VirtualAddress			
	DataDirectories[ENTRY_BASERELOC].Size			
... UNRELATED ...				
Block[0]	VirtualAddress			
	SizeOfBlock			
	type[0]	offset[0]	type[1]	offset[1]
	...	...	...	...
	type[N]	offset[N]	0x0000	
Block[1]	VirtualAddress			
	SizeOfBlock			
	type[0]	offset[0]	type[1]	offset[1]
	...	...	...	...
	type[N]	offset[N]	0x0000	
...	...			
Block[N]	VirtualAddress			
	SizeOfBlock			
	type[0]	offset[0]	type[1]	offset[1]
	...	...	...	...
	type[N]	offset[N]	0x0000	

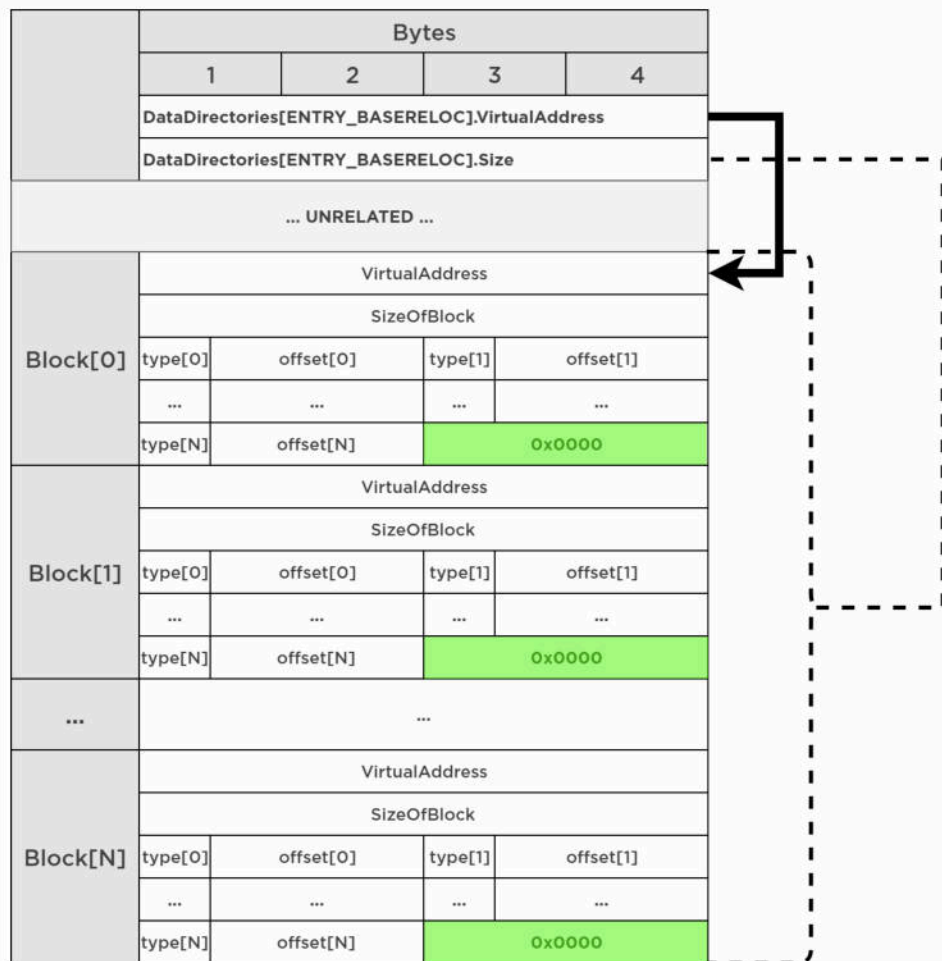
- `virtualAddress` points to first reloc block

How Do Relocations Work?



- `virtualAddress` points to first reloc block
- `Size` is the size, in bytes, of all blocks

How Do Relocations Work?



- `virtualAddress` points to first reloc block
- `Size` is the size, in bytes, of all blocks
- `(uint16_t)0x0000` marks the end of each block

How Do Relocations Work?

	Bytes			
	1	2	3	4
0x00	VirtualAddress			
0x04	SizeOfBlock			
0x08	type[0]	offset[0]	type[1]	offset[1]
0x0C	type[2]	offset[2]	type[3]	offset[3]

How Do Relocations Work?

	Bytes			
	1	2	3	4
0x00	VirtualAddress			
0x04	SizeOfBlock			
0x08	type[0]	offset[0]	type[1]	offset[1]
0x0C	type[2]	offset[2]	type[3]	offset[3]

Virtual Address for start of the block. Offsets are 12-bit, so a block can cover up to 4096 bytes (or 1024 4-byte addresses)

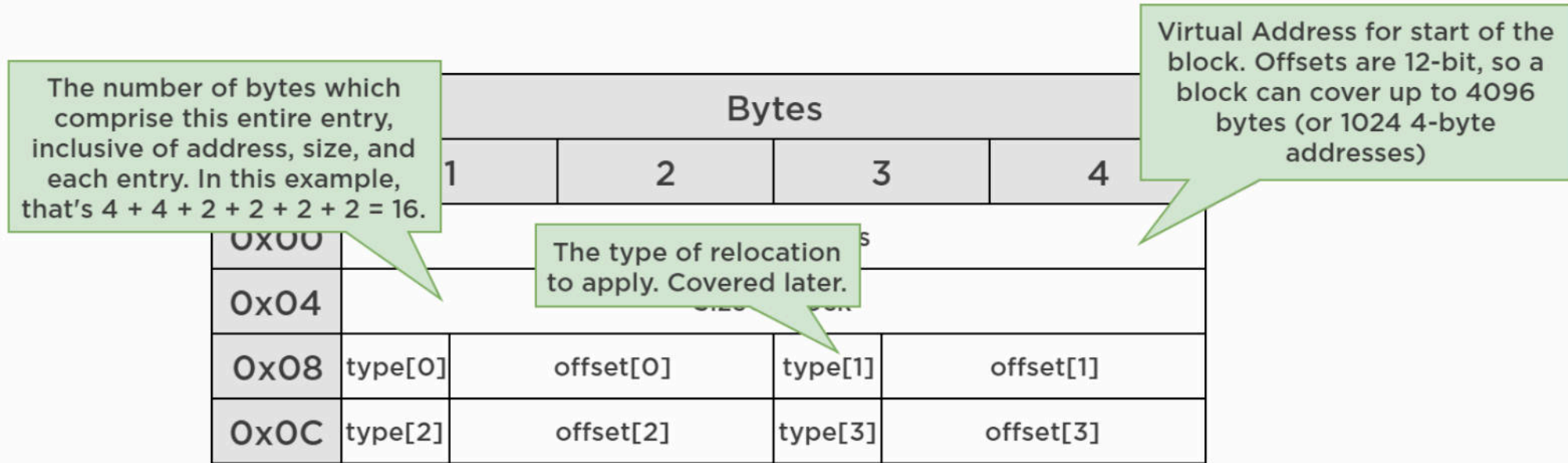
How Do Relocations Work?

The number of bytes which comprise this entire entry, inclusive of address, size, and each entry. In this example, that's $4 + 4 + 2 + 2 + 2 + 2 = 16$.

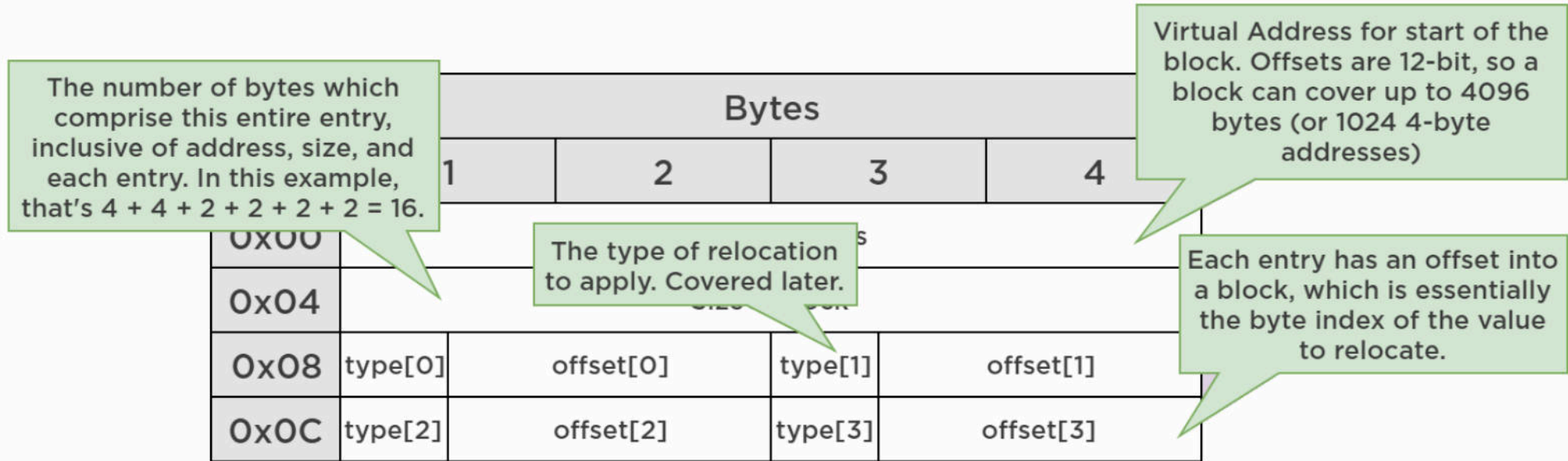
Bytes					
	1	2	3	4	
0x00	VirtualAddress				
0x04	SizeOfBlock				
0x08	type[0]	offset[0]	type[1]	offset[1]	
0x0C	type[2]	offset[2]	type[3]	offset[3]	

Virtual Address for start of the block. Offsets are 12-bit, so a block can cover up to 4096 bytes (or 1024 4-byte addresses)

How Do Relocations Work?



How Do Relocations Work?



How Do Relocations Work?

How Do Relocations Work?

Something like this

```
auto delta = base - desiredBase;
for (auto reloc : relocs) {
    auto block = (base + reloc.VirtualAddress);
    for (auto entry : reloc.entries) {
        auto adr = block + entry.offset;
        if (entry.type == IMAGE_REL_BASED_HIGHLOW) // <- this
            *((uint32_t*)adr) += delta;
        else if (entry.type == IMAGE_REL_BASED_DIR64)
            *((uint64_t*)adr) += delta;
        else if (entry.type == IMAGE_REL_BASED_HIGH)
            *((uint16_t*)adr) += (uint16_t)((delta >> 16) & 0xFFFF);
        else if (entry.type == IMAGE_REL_BASED_LOW)
            *((uint16_t*)adr) += (uint16_t)delta;
    }
}
```

Controlling Relocations

Controlling Relocations

- Relocations simply use a `+=` operation on data at a specified address

Controlling Relocations

- Relocations simply use a `+=` operation on data at a specified address
- The right-hand side of this operation will be `delta`

Controlling Relocations

- Relocations simply use a `+=` operation on data at a specified address
- The right-hand side of this operation will be `delta`
- `delta` is defined as `base - desiredBase`

Controlling Relocations

- Relocations simply use a `+=` operation on data at a specified address
- The right-hand side of this operation will be `delta`
- `delta` is defined as `base - desiredBase`

Conclusion: to abuse relocations, `base` must be preselected, giving a predictable `delta`. This means ASLR must be tricked.

ASLR Preselection

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

Knowing this, I tried `desiredBase` as:

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

Knowing this, I tried `desiredBase` as:

- `0xFFFFFFFF`

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

Knowing this, I tried `desiredBase` as:

- `0xFFFFFFFF`: PE fails to load; invalid header

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

Knowing this, I tried `desiredBase` as:

- `0xFFFFFFFF`: PE fails to load; invalid header
- `0x00000000`

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

Knowing this, I tried `desiredBase` as:

- `0xFFFFFFFF`: PE fails to load; invalid header
- `0x00000000`: PE fails to load; invalid header

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

Knowing this, I tried `desiredBase` as:

- `0xFFFFFFFF`: PE fails to load; invalid header
- `0x00000000`: PE fails to load; invalid header
- `0xFFFF0000`

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

Knowing this, I tried `desiredBase` as:

- `0xFFFFFFFF`: PE fails to load; invalid header
- `0x00000000`: PE fails to load; invalid header
- `0xFFFF0000`: PE loads at base `0x00010000`

ASLR Preselection

- `desiredBase` is the only means of controlling ASLR
- `delta` is dependant on `desiredBase`

Conclusion: because of how `delta` is derived, `desiredBase` must be made to force ASLR mapping at a predetermined address which isn't `desiredBase` itself.

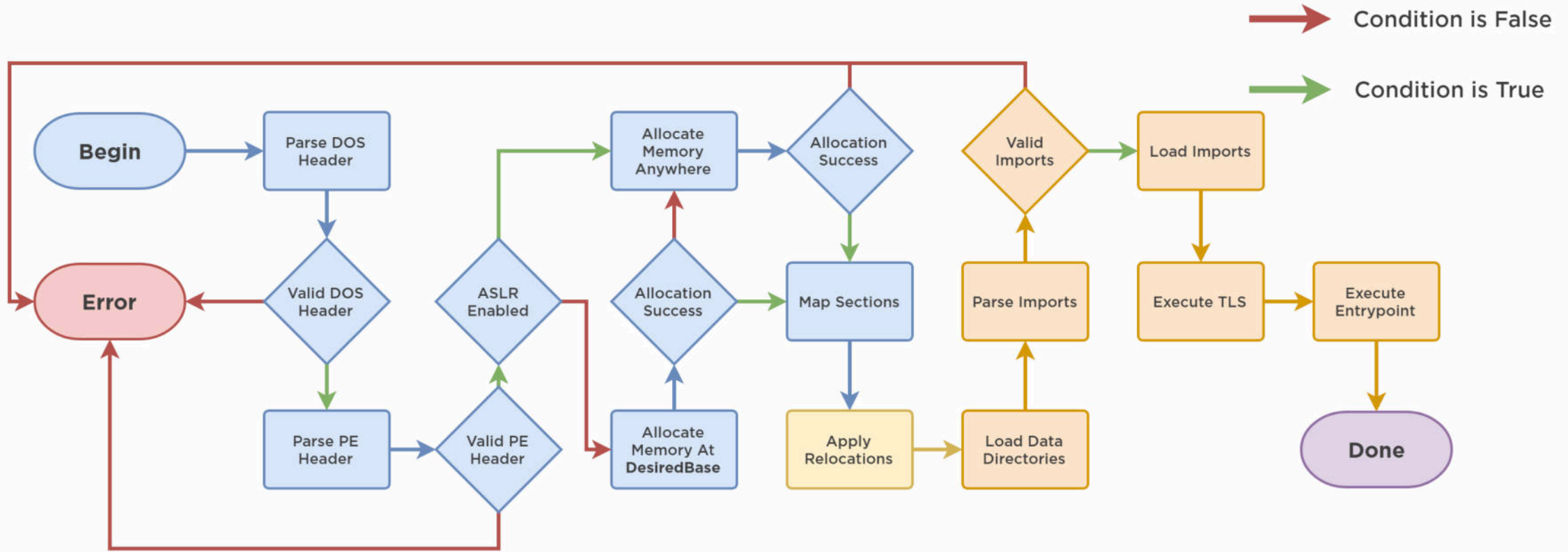
Knowing this, I tried `desiredBase` as:

- `0xFFFFFFFF`: PE fails to load; invalid header
- `0x00000000`: PE fails to load; invalid header
- `0xFFFF0000`: PE loads at base `0x00010000`

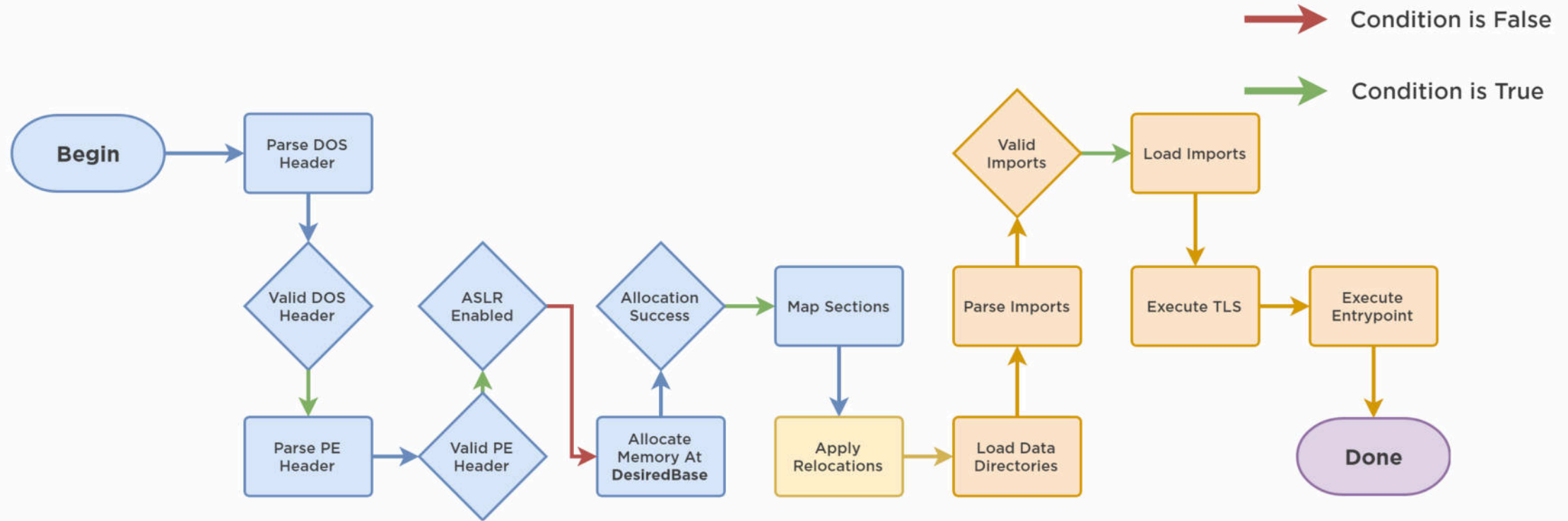
As I later learned, Corkami had already figured all of this out: <https://github.com/corkami/pocs/>

PE Loader Breakdown

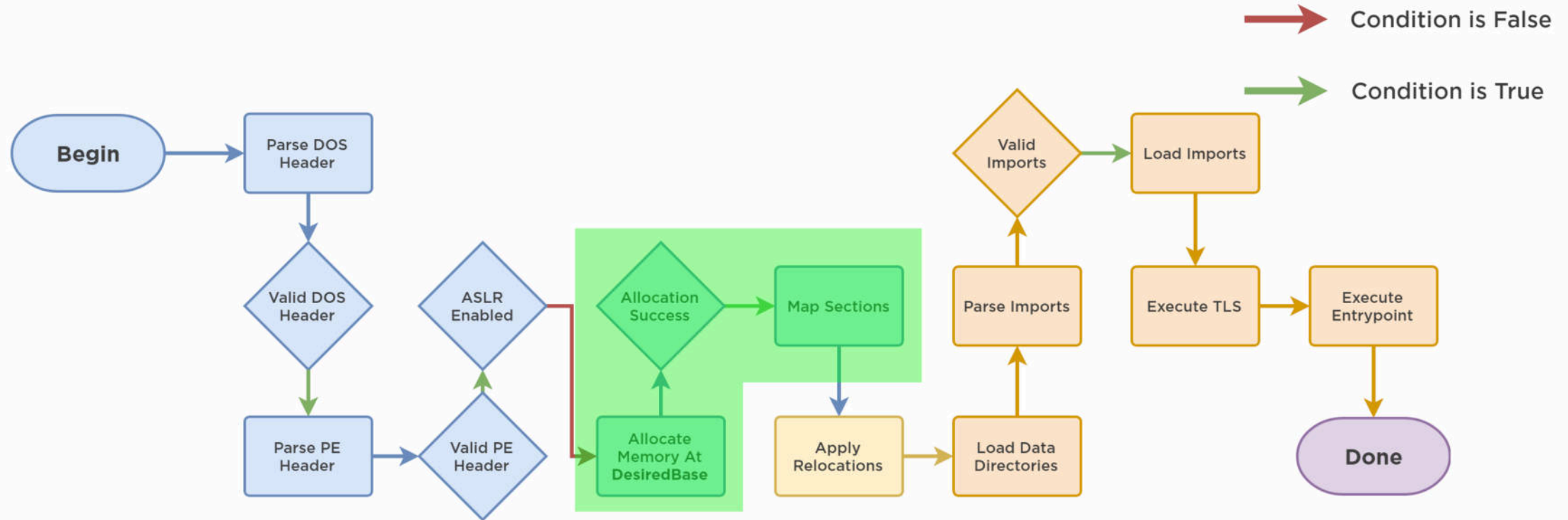
PE Loader Breakdown



PE Loader Breakdown



PE Loader Breakdown



Targets For Relocation Obfuscation

Targets For Relocation Obfuscation

- Import Table is loaded post-reloc

Targets For Relocation Obfuscation

- Import Table is loaded post-reloc
- Many sections are mapped pre-reloc, but not used until execution which is post-reloc

Targets For Relocation Obfuscation

- Import Table is loaded post-reloc
- Many sections are mapped pre-reloc, but not used until execution which is post-reloc
- `entryPoint` isn't used until execution, post-reloc; the memory will be read-only, however, unless DEP is off

Targets For Relocation Obfuscation

- Import Table is loaded post-reloc
- Many sections are mapped pre-reloc, but not used until execution which is post-reloc
- `entryPoint` isn't used until execution, post-reloc; the memory will be read-only, however, unless DEP is off

Conclusion: can mangle imports, code and resource sections, and optionally the `entryPoint` if DEP is off on the target machine.

Targets For Relocation Obfuscation

Targets For Relocation Obfuscation

PE¹⁰¹ a windows executable walk-through

ANGE ALBERTINI
CORKAMI.COM

DISSECTED PE

DOWNLOAD THE PE FILE



HEADER
TECHNICAL DETAILS ABOUT THE EXECUTABLE

SECTIONS
CONTENTS OF THE EXECUTABLE

SIMPLE.EXE

HEADER

TECHNICAL DETAILS ABOUT THE EXECUTABLE

SECTIONS

CONTENTS OF THE EXECUTABLE

DOS HEADER

SHOWS ITS A BINARY

PE HEADER

SHOWS IT'S A MODERN BINARY

OPTIONAL HEADER

EXECUTABLE INFORMATION

DATA DIRECTORIES

PORTS TO EXTRA STRUCTURES (EXPORTS, IMPORTS, ...)

SECTIONS TABLE

DEFINES THE FILE'S SECTIONS

CODE

WHAT IS EXECUTED

IMPORTS

LINK BETWEEN THE EXECUTABLE AND WINDOWS LIBRARIES

DATA

INFORMATION USED BY THE CODE

HEXADECMAL DUMP

ASCII DUMP

FIELDS

VALUES

EXPLANATION

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Magic

AddressOfEntryPoint

ImageBase

SectionAlignment

FileAlignment

MajorSubsystemVersion

SizeOfImage

Subsystem

NumberOfRvaAndSizes

ImportsVA

Sections Table

NAME

VIRTUAL SIZE

VIRTUAL ADDRESS

SIZE OF DATA

PORT TO DATA

CHARACTERISTICS

FOR EACH SECTION A SIZE OF DATA BLOCK IS READ FROM THE FILE AT PORT TO DATA OFFSET

IT WILL BE LOADED IN MEMORY AT ADDRESS PAGEBASE + VIRTUAL ADDRESS IN A VIRTUAL SIZE BLOCK, WITH SPECIFIC CHARACTERISTICS

X86 ASSEMBLY

EQUIVALENT C CODE

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

THIS IS THE WHOLE FILE, HOWEVER, MOST PE FILES CONTAIN MANY ELEMENTS

EXPLANATIONS ARE PROVIDED FOR CONCISENESS

Signature

Machine

NumberOfSections

SizeOfOptionalHeader

Characteristics

Targets For Relocation Obfuscation

PE¹⁰¹ a windows executable walk-through

ANGE ALBERTINI
CORKAMI.COM

DISSECTED PE

DISSECTED PE

HEADER
TECHNICAL DETAILS ABOUT THE EXECUTABLE

SECTIONS
CONTENTS OF THE EXECUTABLE

DATA
INFORMATION USED BY THE CODE

IMPORTS
LINK BETWEEN THE EXECUTABLE AND WINDOWS LIBRARIES

CODE
WHAT EXECUTES

HEXADECMAL DUMP

ASCII DUMP

FIELDS

VALUES

EXPLANATION

IMPORTS STRUCTURES

CONSEQUENCES

STRINGS

text...

ImportsVA

Sections Table

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a data offset. It will be loaded in memory at address: imagebase + virtual address in a virtual size sized block, with specific characteristics.

For each section a size of a data sized block is read from the file at pointer to a

The Final Attack

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`
- Turn ASLR off by flipping a bit in the PE Header

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`
- Turn ASLR off by flipping a bit in the PE Header
- Set `desiredBase` to `0xFFFF0000`

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`
- Turn ASLR off by flipping a bit in the PE Header
- Set `desiredBase` to `0xFFFF0000`
- Loop over data to obfuscate in `uint32_t`-sized chunks, decrementing each by `0x00010000 - 0xFFFF0000` (expected value of `delta`)

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`
- Turn ASLR off by flipping a bit in the PE Header
- Set `desiredBase` to `0xFFFF0000`
- Loop over data to obfuscate in `uint32_t`-sized chunks, decrementing each by `0x00010000 - 0xFFFF0000` (expected value of `delta`)
- Discard original relocations table

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`
- Turn ASLR off by flipping a bit in the PE Header
- Set `desiredBase` to `0xFFFF0000`
- Loop over data to obfuscate in `uint32_t`-sized chunks, decrementing each by `0x00010000 - 0xFFFF0000` (expected value of `delta`)
- Discard original relocations table
- Generate new relocations table containing the location of each decrement done inside of the loop (using `IMAGE_REL_BASED_HIGHLOW`)

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`
- Turn ASLR off by flipping a bit in the PE Header
- Set `desiredBase` to `0xFFFF0000`
- Loop over data to obfuscate in `uint32_t`-sized chunks, decrementing each by `0x00010000 - 0xFFFF0000` (expected value of `delta`)
- Discard original relocations table
- Generate new relocations table containing the location of each decrement done inside of the loop (using `IMAGE_REL_BASED_HIGHLOW`)
- Save new PE file to disk

The Final Attack

Due to the nature of the attack, it works best as a tool which rebuilds regular PE files.

- Load target PE file
- Apply original relocations for `base` of `0x00010000`
- Turn ASLR off by flipping a bit in the PE Header
- Set `desiredBase` to `0xFFFF0000`
- Loop over data to obfuscate in `uint32_t`-sized chunks, decrementing each by `0x00010000 - 0xFFFF0000` (expected value of `delta`)
- Discard original relocations table
- Generate new relocations table containing the location of each decrement done inside of the loop (using `IMAGE_REL_BASED_HIGHLOW`)
- Save new PE file to disk
- ??? profit

ROUND ONE

FIGHT

Testing The Attack

Testing The Attack

- Windows 7

Testing The Attack

- Windows 7 ... works !

Testing The Attack

- Windows 7 ... works !
- Windows 8

Testing The Attack

- **Windows 7** ... works !
- **Windows 8** ... nobody uses this shit

Testing The Attack

- **Windows 7** ... works !
- **Windows 8** ... nobody uses this shit
- **Windows 10**

Testing The Attack

- Windows 7 ... works !
- Windows 8 ... nobody uses this shit
- Windows 10



Testing The Attack

- Windows 7 ... works
- Windows 8 ... nobody uses this shit
- Windows 10



THE END

Exploring New Terrain

Exploring New Terrain

- Embed PE copies for all possible base addresses

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big 🍆
- Tweaking ASLR Configuration

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big 🍆
- Tweaking ASLR Configuration ... works!

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big 🍆
- Tweaking ASLR Configuration ... works!
 - Set `Mandatory ASLR` to `On`

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big 🍆
- Tweaking ASLR Configuration ... works!
 - Set `Mandatory ASLR` to `On`
 - Set `Bottom-Up ASLR` to `Off`

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big 🍆
- Tweaking ASLR Configuration ... works!
 - Set **Mandatory ASLR** to **On**
 - Set **Bottom-Up ASLR** to **Off**

```
[HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\
Image File Execution Options\NAME_OF_EXE]
"MitigationAuditOptions"=hex:00,00,00,00,00,00,00,00,\
                                00,00,00,00,00,00,00,00
"MitigationOptions"=hex:00,01,22,00,00,00,00,00,\
                        00,00,00,00,00,00,00,00
```

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big 🍆
- Tweaking ASLR Configuration ... works!
 - Set **Mandatory ASLR** to **On**
 - Set **Bottom-Up ASLR** to **Off**

```
[HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\
Image File Execution Options\NAME_OF_EXE]
"MitigationAuditOptions"=hex:00,00,00,00,00,00,00,00,\
                                00,00,00,00,00,00,00,00
"MitigationOptions"=hex:00,01,22,00,00,00,00,00,\
                        00,00,00,00,00,00,00,00
```

Conclusion: it can work on Windows 10

Exploring New Terrain

- Embed PE copies for all possible base addresses ... way too big 🍆
- Tweaking ASLR Configuration ... works!
 - Set **Mandatory ASLR** to **On**
 - Set **Bottom-Up ASLR** to **Off**

```
[HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\
Image File Execution Options\NAME_OF_EXE]
"MitigationAuditOptions"=hex:00,00,00,00,00,00,00,00,\
                                00,00,00,00,00,00,00,00
"MitigationOptions"=hex:00,01,22,00,00,00,00,00,\
                        00,00,00,00,00,00,00,00
```

Conclusion: it can work on Windows 10, but I don't like it

A New Hope

A New Hope

```
nick@neutrino: ~  
nick@neutrino:~$ listdlls.exe | grep -v Command | grep -v pid | grep basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
nick@neutrino:~$
```

A New Hope

```
nick@neutrino: ~  
nick@neutrino:~$ listdlls.exe | grep -v Command | grep -v pid | grep basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
nick@neutrino:~$
```

A New Hope

```
nick@neutrino: ~  
nick@neutrino:~$ listdlls.exe | grep -v Command | grep -v pid | grep basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
0x000000000000f000 0x6000 C:\basetest.exe  
nick@neutrino:~$ mv /mnt/c/basetest.exe /mnt/c/old.exe  
nick@neutrino:~$ cat /mnt/c/old.exe >/mnt/c/basetest.exe  
nick@neutrino:~$
```

A New Hope

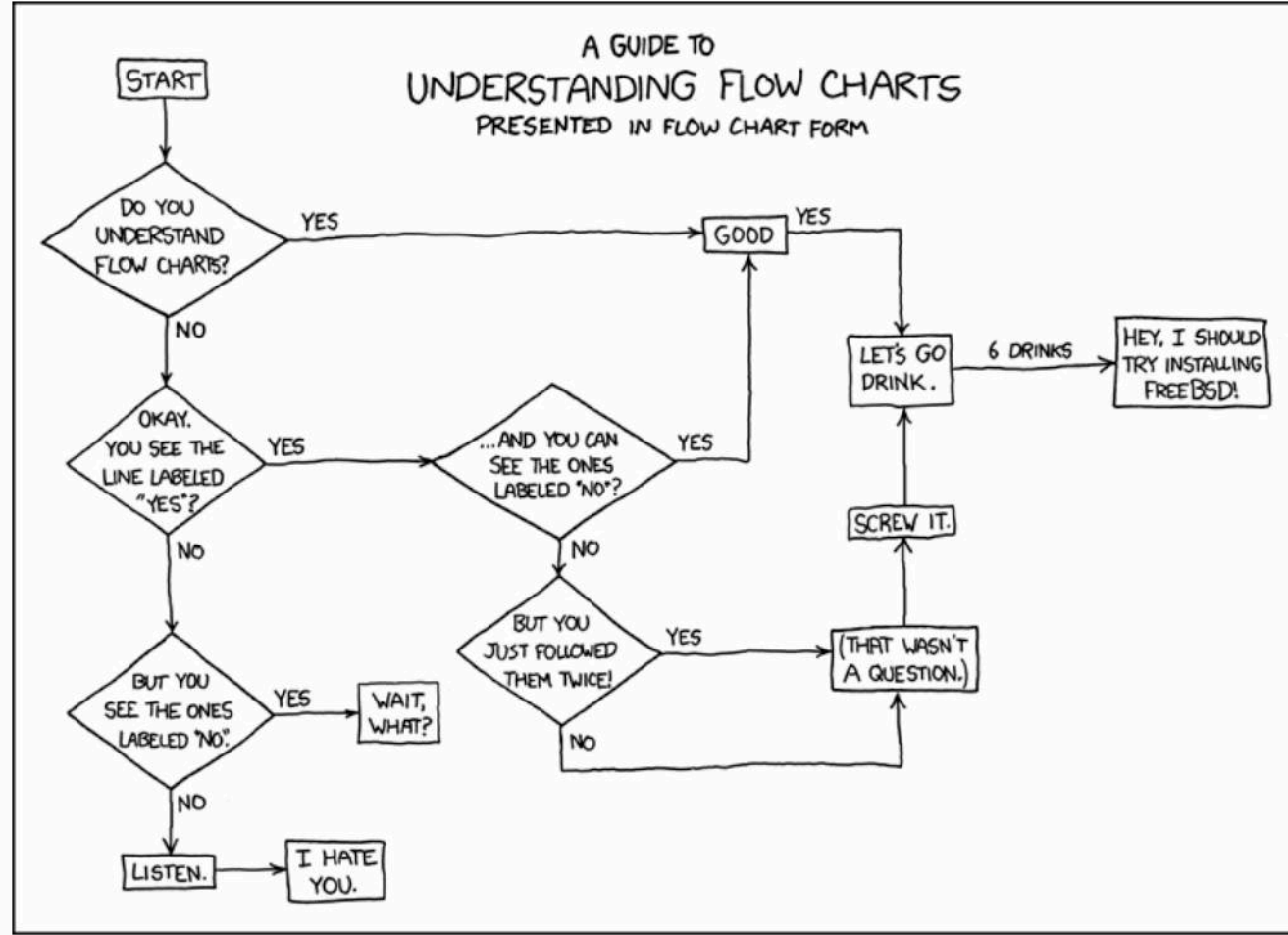
```
nick@neutrino: ~  
nick@neutrino:~$ listdlls.exe | grep -v Command | grep -v pid | grep basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
0x00000000f0000  0x6000  C:\basetest.exe  
nick@neutrino:~$ mv /mnt/c/basetest.exe /mnt/c/old.exe  
nick@neutrino:~$ cat /mnt/c/old.exe >/mnt/c/basetest.exe  
nick@neutrino:~$ listdlls.exe | grep -v Command | grep -v pid | grep basetest.exe  
0x00000000a0000  0x6000  C:\basetest.exe  
0x00000000a0000  0x6000  C:\basetest.exe  
0x00000000a0000  0x6000  C:\basetest.exe  
0x00000000a0000  0x6000  C:\basetest.exe  
0x00000000a0000  0x6000  C:\basetest.exe  
0x00000000a0000  0x6000  C:\basetest.exe  
0x00000000a0000  0x6000  C:\basetest.exe  
nick@neutrino:~$
```

A New Hope

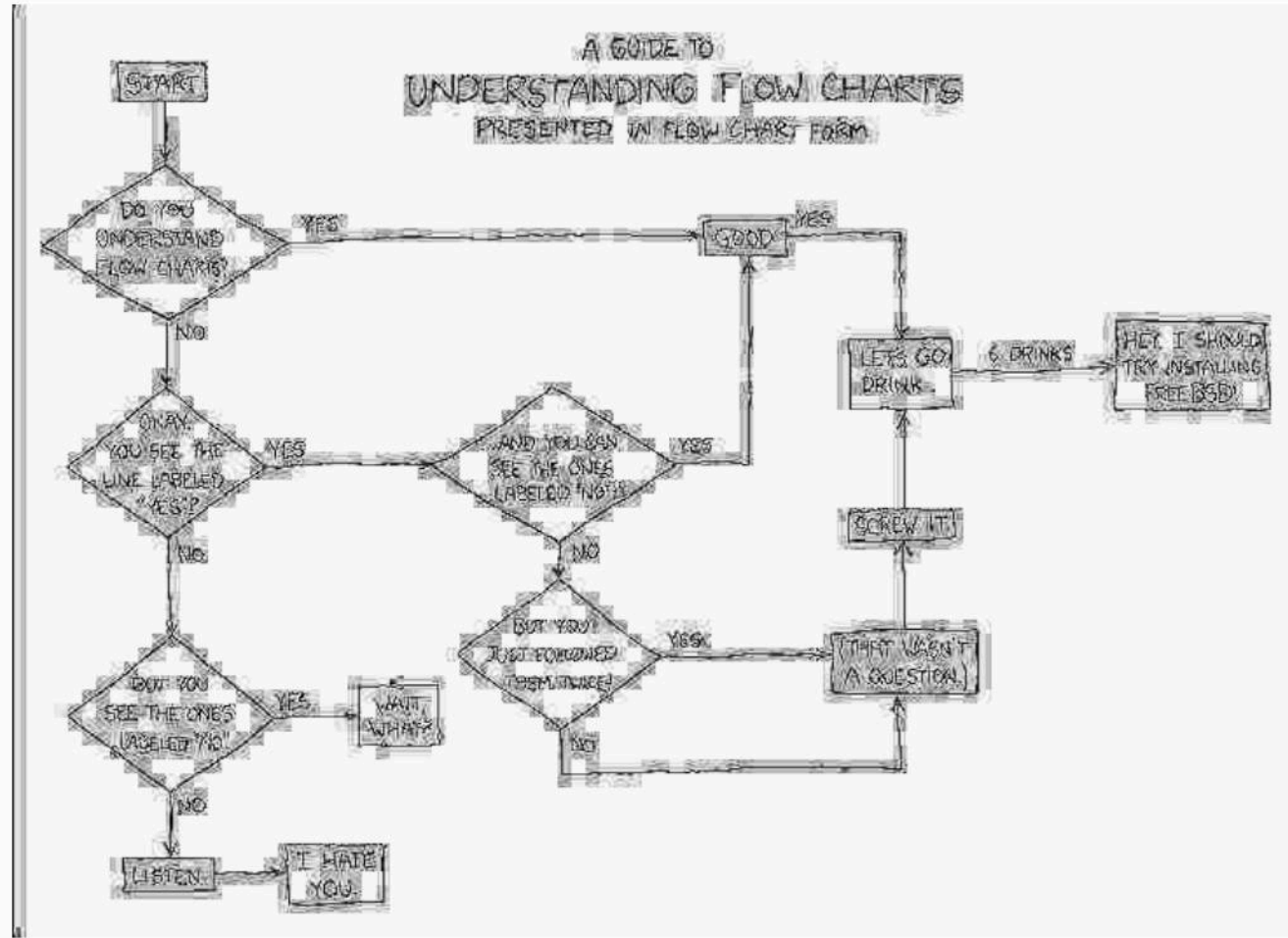
```
nick@neutrino: ~  
nick@neutrino:~$ listdlls.exe | grep -v Command | grep -v pid | grep basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
0x00000000f0000 0x6000 C:\basetest.exe  
nick@neutrino:~$ mv /mnt/c/basetest.exe /mnt/c/old.exe  
nick@neutrino:~$ cat /mnt/c/old.exe >/mnt/c/basetest.exe  
nick@neutrino:~$ listdlls.exe | grep -v Command | grep -v pid | grep basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
0x00000000a0000 0x6000 C:\basetest.exe  
nick@neutrino:~$
```

Preselection via File Mapping Invalidation

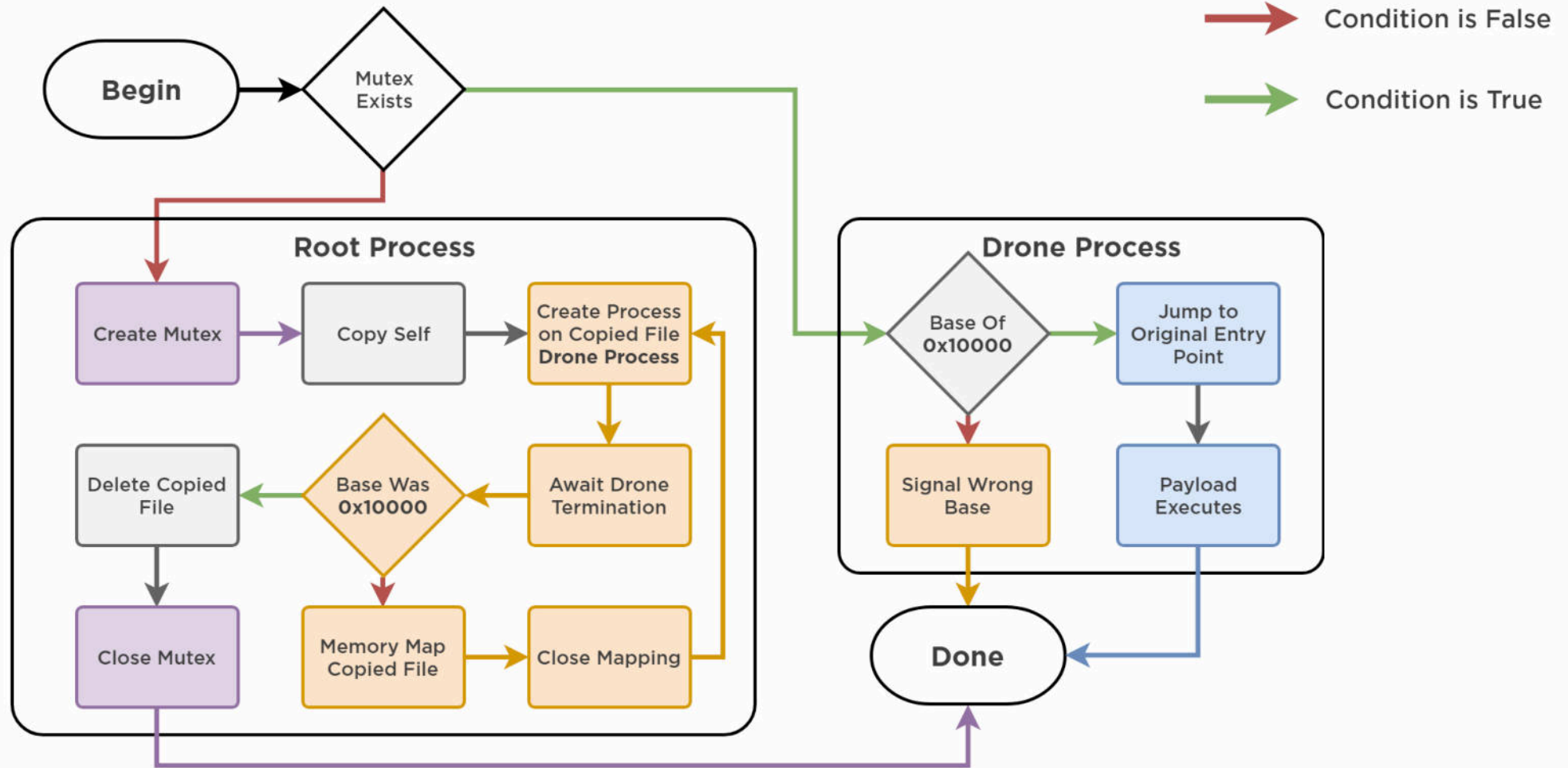
Preselection via File Mapping Invalidation



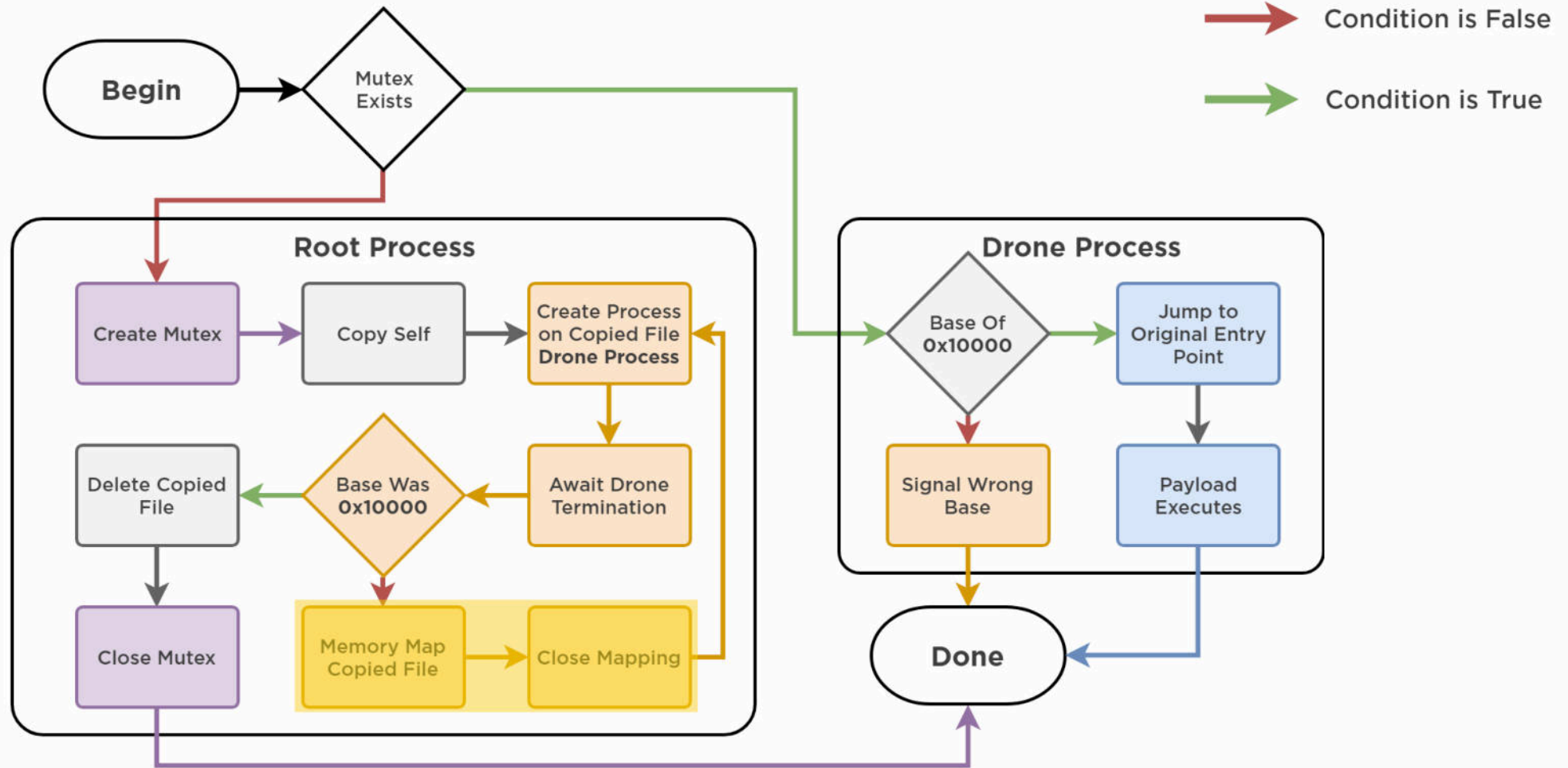
Preselection via File Mapping Invalidation



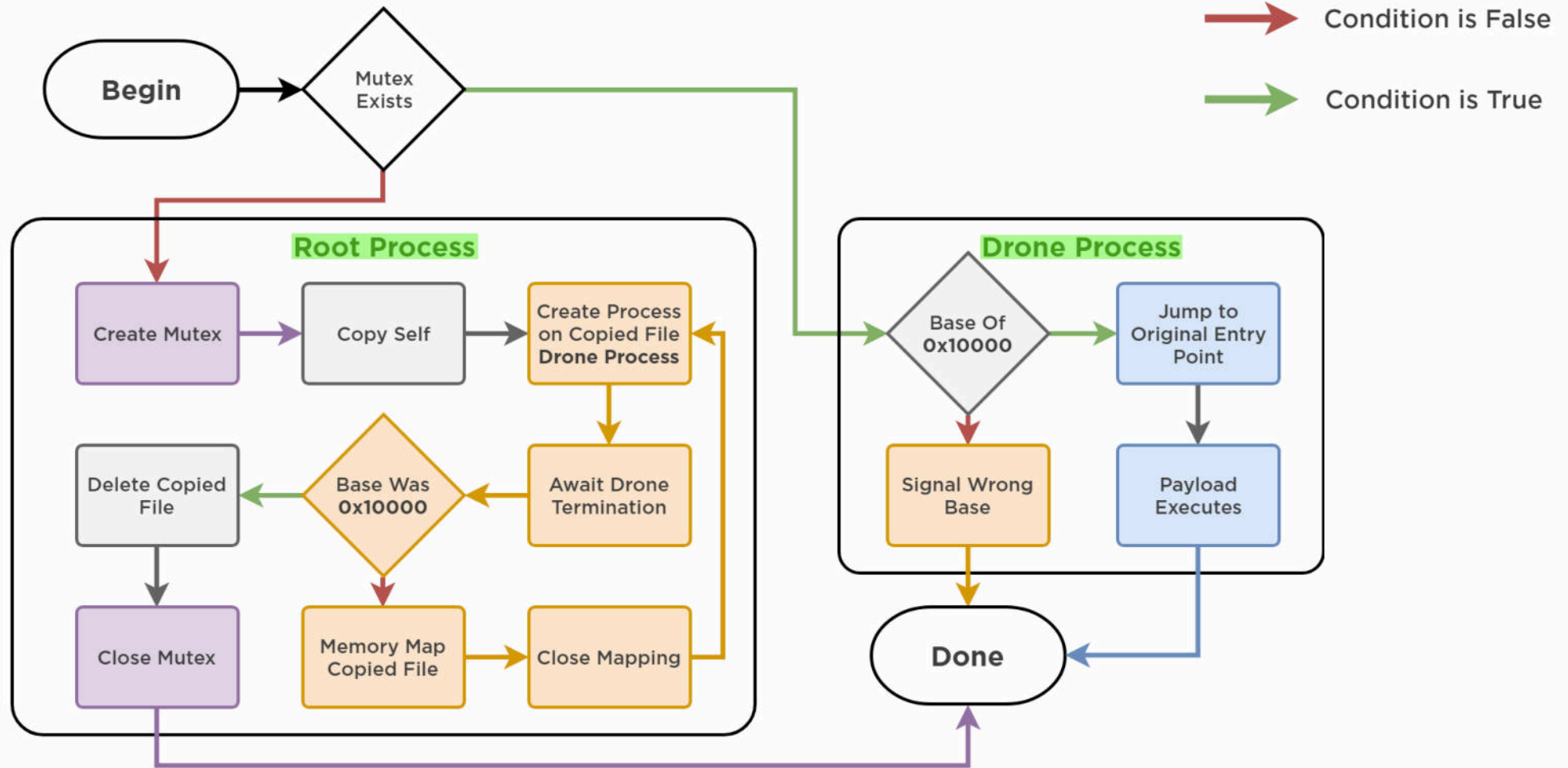
Preselection via File Mapping Invalidation



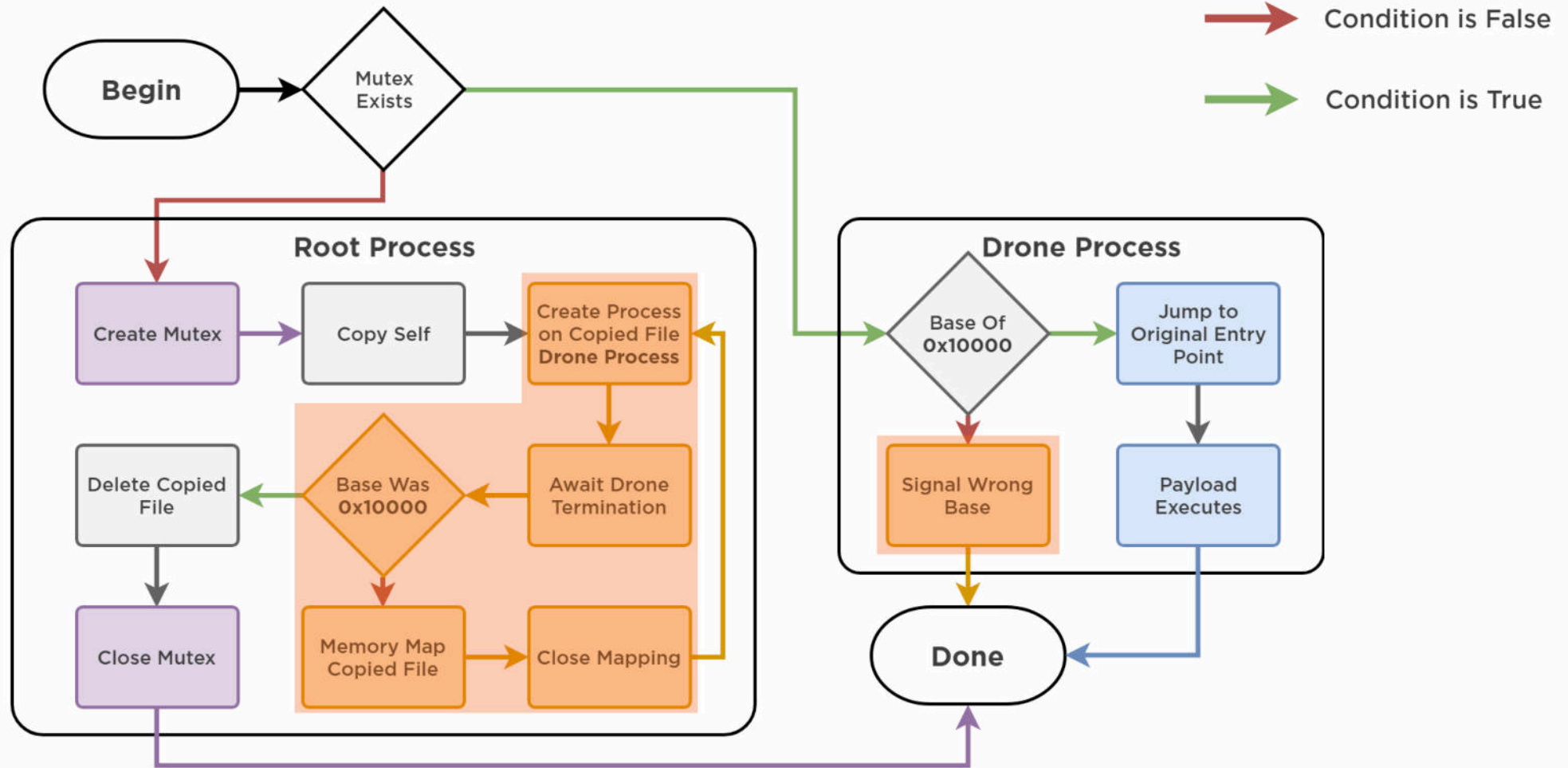
Preselection via File Mapping Invalidation



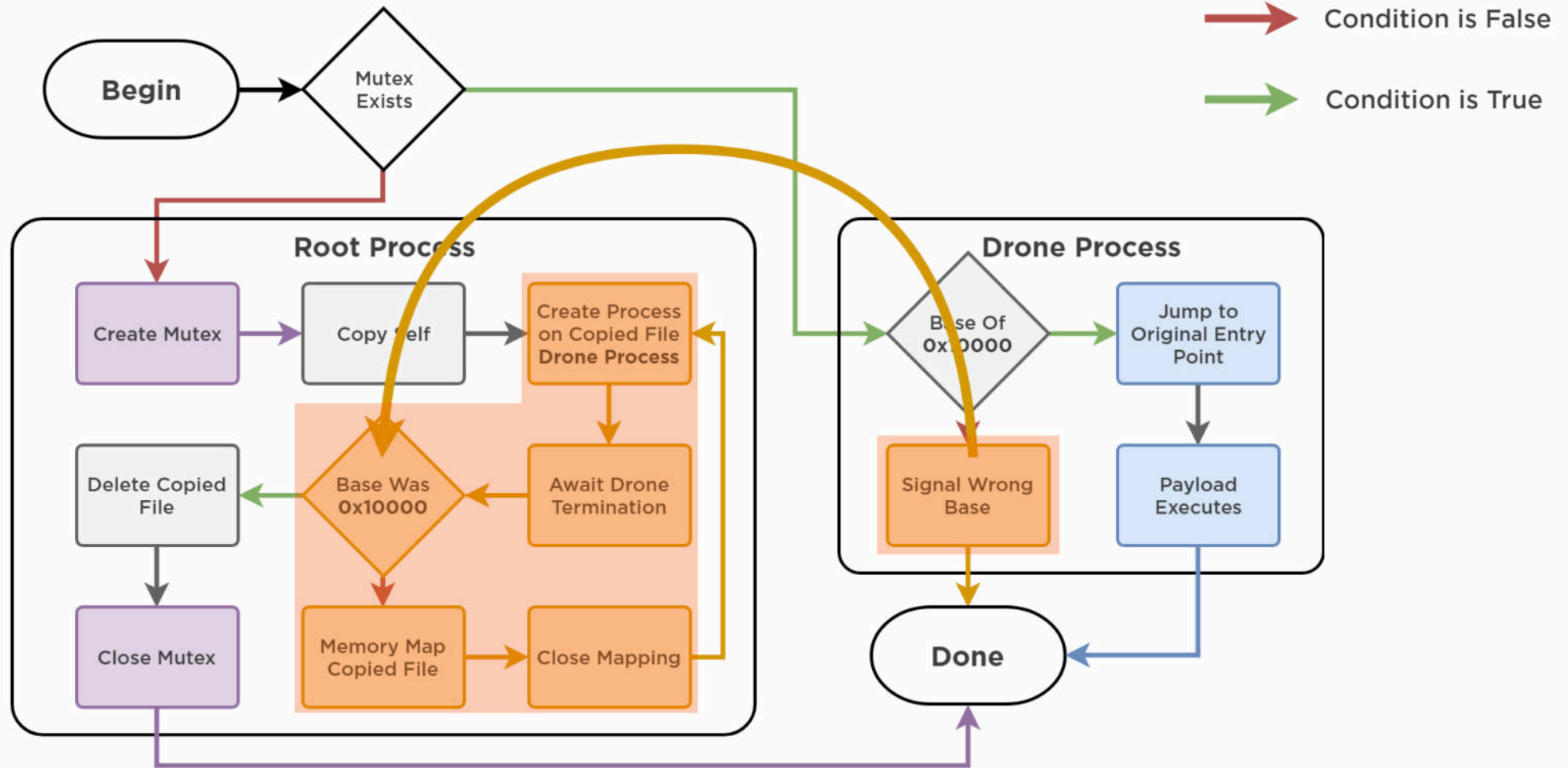
Preselection via File Mapping Invalidation



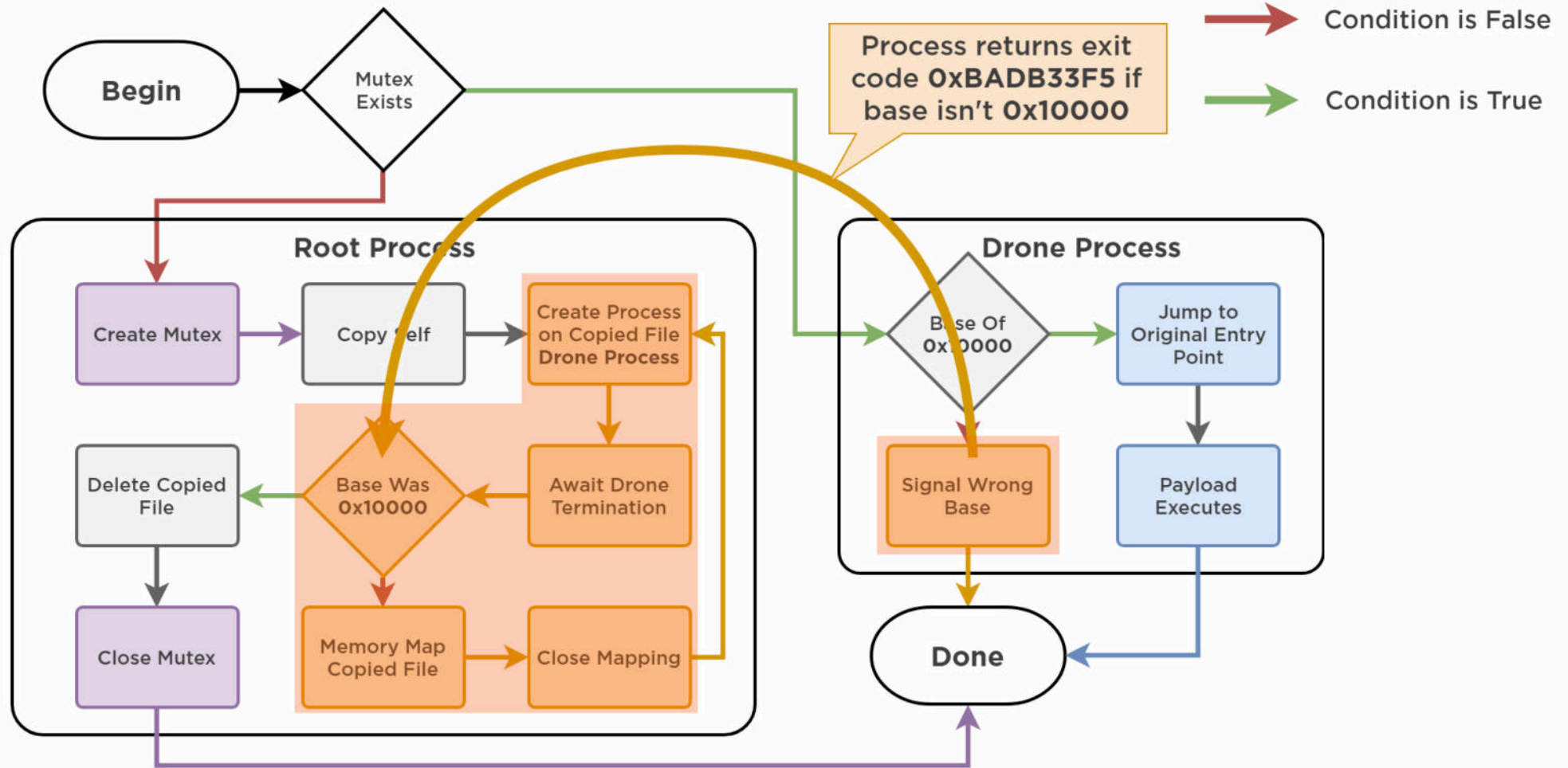
Preselection via File Mapping Invalidation



Preselection via File Mapping Invalidation



Preselection via File Mapping Invalidation



Weaponization

Weaponization

The tool must:

Weaponization

The tool must:

- Create a new section with enough room for the code

Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section

Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section
- Inform the embedded code of the true `EntryPoint`

Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section
- Inform the embedded code of the true `EntryPoint`
- Overwrite `EntryPoint` to point to the embedded code

Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section
- Inform the embedded code of the true `EntryPoint`
- Overwrite `EntryPoint` to point to the embedded code



Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section
- Inform the embedded code of the true `EntryPoint`
- Overwrite `EntryPoint` to point to the embedded code

For this to work, the ASLR preselection code must be:



Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section
- Inform the embedded code of the true `EntryPoint`
- Overwrite `EntryPoint` to point to the embedded code



For this to work, the ASLR preselection code must be:

- Position-agnostic

Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section
- Inform the embedded code of the true `EntryPoint`
- Overwrite `EntryPoint` to point to the embedded code



For this to work, the ASLR preselection code must be:

- Position-agnostic
- Generically embeddable in any PE

Weaponization

The tool must:

- Create a new section with enough room for the code
- Embed the code inside of this new section
- Inform the embedded code of the true `EntryPoint`
- Overwrite `EntryPoint` to point to the embedded code



For this to work, the ASLR preselection code must be:

- Position-agnostic
- Generically embeddable in any PE


```

LDR_GET_PROC(f_GetModuleHandle, kernel32Handle, HMODULE(__stdcall *) (PUCHAR), LDR_STR_SIG('G', 'e', 't', 'M', 'o', 'd', 'u', 'l', 'e', 'H', 'a', 'n', 'd', 'l', 'e', 'A'));
LDR_GET_PROC(f_CreateMutexA, kernel32Handle, HANDLE(__stdcall *) (PVOID, BOOL, LPCSTR), LDR_STR_SIG('C', 'r', 'e', 'a', 't', 'e', 'M', 'u', 't', 'e', 'x', 'A'));
LDR_GET_PROC(f_GetLastError, kernel32Handle, DWORD(__stdcall *) (), LDR_STR_SIG('G', 'e', 't', 'E', 'r', 'r', 'o', 'r'));
LDR_GET_PROC(f_CloseHandle, kernel32Handle, BOOL(__stdcall *) (HANDLE), LDR_STR_SIG('C', 'l', 'o', 's', 'e', 'H', 'a', 'n', 'd', 'l', 'e'));
LDR_GET_PROC(f_GetCurrentProcessId, kernel32Handle, DWORD(__stdcall *) (), LDR_STR_SIG('G', 'e', 't', 'C', 'u', 'r', 'r', 'e', 'n', 't', 'P', 'r', 'o', 'c', 'e', 's', 's', 'I', 'd'));
LDR_GET_PROC(f_GetModuleFileNameW, kernel32Handle, DWORD(__stdcall *) (HMODULE, LPWSTR, DWORD), LDR_STR_SIG('G', 'e', 't', 'M', 'o', 'd', 'u', 'l', 'e', 'F', 'i', 'l', 'e', 'N', 'a', 'm', 'e', 'W'));
LDR_GET_PROC(f_CreateProcessW, kernel32Handle, BOOL(__stdcall *) (LPCWSTR, LPWSTR, PVOID, PVOID, BOOL, DWORD, LPVOID, LPCWSTR, LPSTARTUPINFO, LPPROCESS_INFORMATION), LDR_STR_SIG('C', 'r', 'e', 'a', 't', 'e', 'P', 'r', 'o', 'c', 'e', 's', 's', 'W'));
LDR_GET_PROC(f_WaitForSingleObject, kernel32Handle, DWORD(__stdcall *) (HANDLE, DWORD), LDR_STR_SIG('W', 'a', 'i', 't', 'F', 'o', 'r', 'S', 'i', 'n', 'g', 'l', 'e', 'O', 'b', 'j', 'e', 'c', 't'));
LDR_GET_PROC(f_GetExitCodeProcess, kernel32Handle, BOOL(__stdcall *) (HANDLE, LPDWORD), LDR_STR_SIG('G', 'e', 't', 'E', 'x', 'i', 't', 'C', 'o', 'd', 'e', 'P', 'r', 'o', 'c', 'e', 's', 's'));
LDR_GET_PROC(f_CreateFileW, kernel32Handle, HANDLE(__stdcall *) (LPCWSTR, DWORD, DWORD, LPSECURITY_ATTRIBUTES, DWORD, DWORD, HANDLE), LDR_STR_SIG('C', 'r', 'e', 'a', 't', 'e', 'F', 'i', 'l', 'e', 'W'));
LDR_GET_PROC(f_CreateFileMappingW, kernel32Handle, HANDLE(__stdcall *) (HANDLE, LPSECURITY_ATTRIBUTES, DWORD, DWORD, DWORD, LPCWSTR), LDR_STR_SIG('C', 'r', 'e', 'a', 't', 'e', 'F', 'i', 'l', 'e', 'M', 'a', 'p', 'p', 'i', 'n', 'g', 'W'));
LDR_GET_PROC(f_MapViewOfFileEx, kernel32Handle, LPVOID(__stdcall *) (HANDLE, DWORD, DWORD, DWORD, SIZE_T, LPVOID), LDR_STR_SIG('M', 'a', 'p', 'V', 'i', 'e', 'w', 'O', 'f', 'F', 'i', 'l', 'e', 'E', 'x'));
LDR_GET_PROC(f_UnmapViewOfFile, kernel32Handle, BOOL(__stdcall *) (LPCVOID), LDR_STR_SIG('U', 'n', 'm', 'a', 'p', 'V', 'i', 'e', 'w', 'O', 'f', 'F', 'i', 'l', 'e'));
LDR_GET_PROC(f_CopyFileW, kernel32Handle, BOOL(__stdcall *) (LPCWSTR, LPCWSTR, BOOL), LDR_STR_SIG('C', 'o', 'p', 'y', 'F', 'i', 'l', 'e', 'W'));
LDR_GET_PROC(f_DeleteFileW, kernel32Handle, BOOL(__stdcall *) (LPCWSTR), LDR_STR_SIG('D', 'e', 'l', 'e', 't', 'e', 'F', 'i', 'l', 'e', 'W'));
LDR_GET_PROC(f_lstrcatW, kernel32Handle, LPWSTR(__stdcall *) (LPWSTR, LPCWSTR), LDR_STR_SIG('l', 's', 't', 'r', 'c', 'a', 't', 'W'));
LDR_GET_PROC(f_ExitProcess, kernel32Handle, VOID(__stdcall *) (DWORD), LDR_STR_SIG('E', 'x', 'i', 't', 'P', 'r', 'o', 'c', 'e', 's', 's'));

// define strings
DEFINE_STR(const char*, mutexName, 'r', 'e', 'l', 'o', 'c', 'a', 'l', 'p', 'a', 'c', 'k', '_', 'm', 'u', 't');
DEFINE_STR(const wchar_t*, copySuffix, '2', 0x00, '.', 0x00, 'e', 0x00, 'x', 0x00, 'e', 0x00);

```

Weaponization

Weaponization

It works!

Weaponization

It works!

Caveats

Weaponization

It works!

Caveats

- It can be slow, averaging of 200 iterations to land

Weaponization

It works!

Caveats

- It can be slow, averaging of 200 iterations to land
- Imports can't be obfuscated

Weaponization

It works!

Caveats

- It can be slow, averaging of 200 iterations to land
- Imports can't be obfuscated

Advantages

Weaponization

It works!

Caveats

- It can be slow, averaging of 200 iterations to land
- Imports can't be obfuscated

Advantages

- Base can be anything, not just `0x00010000`!

Weaponization

It works!

Caveats

- It can be slow, averaging of 200 iterations to land
- Imports can't be obfuscated

Advantages

- Base can be anything, not just `0x00010000`!
- As a side effect, some form of symbolic execution is needed to discover the intended base in order to fix up the file for analysis.

ROUND

TWO

FIGHT

Use Cases

Use Cases

- Annoying analysts

Use Cases

- Annoying analysts
- Breaking automated static analysis systems

Use Cases

- Annoying analysts
- Breaking automated static analysis systems
- Breaking tools

Use Cases

- Annoying analysts
- Breaking automated static analysis systems
- Breaking tools
- Breaking AV Parsers

Potential Improvements

Potential Improvements

- More obfuscations

Potential Improvements

- More obfuscations
 - New targets

Potential Improvements

- More obfuscations
 - New targets
 - Multiple passes

Potential Improvements

- More obfuscations
 - New targets
 - Multiple passes
- Header Scrambling

Potential Improvements

- More obfuscations
 - New targets
 - Multiple passes
- Header Scrambling
- Combining with runtime packers

Potential Improvements

- More obfuscations
 - New targets
 - Multiple passes
- Header Scrambling
- Combining with runtime packers
- Support for 64bit binaries

Potential Improvements

- More obfuscations
 - New targets
 - Multiple passes
- Header Scrambling
- Combining with runtime packers
- Support for 64bit binaries
- Support for DLLs

Potential Improvements

- More obfuscations
 - New targets
 - Multiple passes
- Header Scrambling
- Combining with runtime packers
- Support for 64bit binaries
- Support for DLLs
- Selective obfuscations

THE END

Find Me

<https://nickcano.com>

<https://github.com/nickcano>

<https://twitter.com/nickcano93>

<https://nostarch.com/gamehacking>

<https://pluralsight.com/authors/nick-cano>

Resources

<https://msdn.microsoft.com/en-us/library/ms809762.aspx>

<https://github.com/corkami/pocs/tree/master/PE>

Source Code

<https://github.com/nickcano/RelocBonus>

<https://github.com/nickcano/RelocBonusSlides>

